

BuildTrack

Software Requirements Document

How BuildTrack will be built: architecture, data model, API, business logic, mobile and web apps, security, and a milestone-by-milestone build plan.
Companion to the PRD v1.0.

Document	Software Requirements Document (SRD)
Product	BuildTrack (working name)
Version	1.0 — Draft for review
Date	27 September 2026
Author / builder	Izuchukwu (solo developer)
Based on	BuildTrack PRD v1.0 (draft), 27 September 2026
Scope	Building the product. Hosting and deployment are excluded for now.

Contents

0 Document control	3
1 Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 References	4
1.4 Decisions confirmed for this SRD	4
1.5 Deferred (kept configurable)	5
2 System architecture	6
2.1 Overview	6
2.2 Architecture principles	6
2.3 Technology stack	7
2.4 Request lifecycle	7
3 Codebase structure	9
3.1 Monorepo layout	9
3.2 Package rules	9
3.3 Central constants	9
4 Data design	11
4.1 Prisma setup	11
4.2 Entity overview	11
4.3 Modelling rules	12
4.4 Immutability enforcement	12
4.5 File storage layout	13
5 Authentication & sessions	14
5.1 Approach	14
5.2 Requirements	14
5.3 Request context	14
6 Authorisation	15
6.1 Model	15
6.2 Scoping rules	15
6.3 Action catalogue (excerpt)	15
6.4 Database-level protection	16
7 API design	17
7.1 Conventions	17
7.2 Endpoint catalogue	17
8 Domain logic & state machines	23
8.1 Plan lifecycle	23
8.2 Stage lifecycle	23
8.3 Payment lifecycle	24
8.4 Dispute lifecycle	24
8.5 Change request lifecycle	25
8.6 Project lifecycle	25
8.7 Progress calculation	25
8.8 Timers (ScheduledJob)	25
8.9 Domain events	26
9 Proof & media pipeline	27
9.1 Capture (mobile)	27
9.2 Processing	27
9.3 Geofence	27
9.4 Upload flow	27
9.5 Minimum-proof check	27
10 Mobile app (React Native / Expo)	28
10.1 Foundations	28

10.2	Navigation by role	28
10.3	Offline architecture	28
10.4	Push notifications & deep links	29
11	Web app (Next.js)	30
11.1	Route map	30
11.2	Rendering rules	30
12	Chat & realtime	31
12.1	Design	31
12.2	Channel security	31
12.3	Features	31
13	Notifications engine	32
13.1	Pipeline	32
13.2	Rules	32
14	Integrations (adapter interfaces)	33
14.1	Name matching	33
15	Rewards engine	34
16	Billing & entitlements	35
17	Security & privacy	36
18	Quality, testing & local development	37
18.1	Performance targets	37
18.2	Testing strategy	37
18.3	Local development	37
18.4	Observability	37
19	Build plan	38
20	Open items	40
A	Appendix A — Prisma schema (models)	41
B	Appendix B — Enums	52
C	Appendix C — Notification event catalogue	54
D	Appendix D — Error codes	56

0 Document control

Version history

Version	Date	Author	Changes
1.0 (draft)	27 Sep 2026	Izuchukwu	First SRD, derived from PRD v1.0 and the technical decisions agreed after it.

How to read this document

- **Sections 1–3:** what is being built, the architecture and how the code is organised.
- **Sections 4–7:** the data model, sign-in, permissions and the API that both apps use.
- **Sections 8–16:** the business logic (state machines, timers), proof and media, the mobile and web apps, chat, notifications, integrations, rewards and billing.
- **Sections 17–18:** security, privacy, testing and quality.
- **Section 19:** the build plan — milestones in the order to build them.
- **Appendices:** the full Prisma schema, enums, the notification event catalogue and error codes.

Conventions

Convention	Meaning
must / should / may	Mandatory / strongly recommended / optional.
SR-xxx-nn	Software requirement ID. Where relevant it references the PRD functional requirement it implements (e.g. FR-PAY-05).
Release tags	   as in the PRD. The data model already includes v1.1 fields so no painful migrations are needed later.
code	Names of tables, fields, endpoints, enums and functions.
TBD	Deliberately undecided; the design keeps it configurable.

1 Introduction

1.1 Purpose

The PRD defines *what* BuildTrack does. This SRD defines *how* it is built, precisely enough that a single developer can start coding from Milestone 0 and keep every decision consistent across the web app, the mobile app and the backend.

1.2 Scope

In scope

- System architecture and code organisation
- Data model (Prisma / PostgreSQL)
- Authentication, sessions and authorisation
- REST API used by both web and mobile
- State machines, timers and business rules
- Proof capture, media handling, offline sync
- Chat and realtime, notifications
- Integration interfaces (verification, SMS, email, push, billing)
- Rewards and billing logic
- Security, privacy, testing, local development
- Build plan and milestones

Out of scope (for now)

- Hosting, cloud provider (AWS / Azure), deployment and CI/CD infrastructure
- Choice of third-party providers — interfaces only
- Subscription prices
- Reward period and recognition-point cap values
- "Payment due before a stage starts" option
- v2 features (in-app payments, platform dispute rulings, multi-contractor projects) — only noted where the design must allow for them

1.3 References

- BuildTrack PRD v1.0 (draft) — all FR-* IDs refer to it.
- Construction Tracking Competitor Research (ideation phase).
- Supabase documentation: Realtime Authorization (private channels, RLS on `realtime.messages`); Auth Hooks — Send SMS Hook (custom SMS provider for phone OTP).

1.4 Decisions confirmed for this SRD

Topic	Decision
Builder	One developer builds everything.
Stack	React + Next.js (web and API), Prisma, Supabase, React Native (mobile).
Backend	One backend and one database shared by the web and mobile apps.
Hosting	Not decided (AWS or Azure later). The design must not depend on either.
All Tracker	BuildTrack is fully separate from SPANR for now (own database, own accounts).
Permissions	PRD Section 5.3 defaults confirmed.

Topic	Decision
Gallery uploads	Allowed, but labelled " Not captured in app " and do not count toward a stage's minimum proof.
Bank-detail change	Payments pause for 48 hours .
Video	Maximum 60 seconds per clip.
Timeline	None set; the build plan is ordered by milestones, not dates.

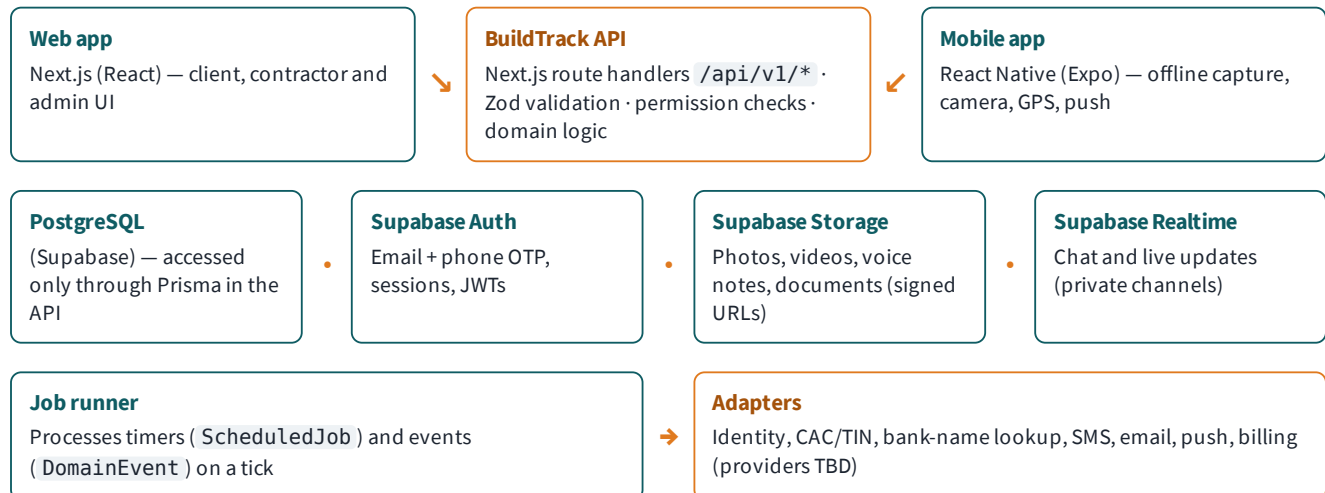
1.5 Deferred (kept configurable)

- **Rewards:** period (monthly vs per project) and recognition-point cap → stored in `Contractor.rewardConfig`; engine reads them, no hard-coded values.
- **Payment due before a stage starts:** not built in v1.0. The `PaymentDueTrigger` enum can gain a `BEFORE_STAGE_START` value later without restructuring.
- **Providers:** every external service sits behind an interface (Section 14); concrete providers are chosen later.

2 System architecture

2.1 Overview

One API, one database. The Next.js app serves the web front end *and* the REST API; the React Native app calls the same API. Supabase provides the PostgreSQL database, authentication, file storage and realtime.



2.2 Architecture principles

#	Principle	Why / consequence
1	The API is the only door to the database.	Web and mobile never query tables directly. All permission checks live in one place. Prisma connects with full privileges, so Supabase row-level security does <i>not</i> protect Prisma queries — the API must.
2	Two narrow exceptions use Supabase directly	(a) file uploads/downloads via short-lived signed URLs issued by the API; (b) Realtime private channels for chat and live updates, protected by RLS policies on <code>realtime.messages</code> .
3	Business rules live in a pure TypeScript package (<code>@bt/core</code>)	State machines, progress maths and point rules are plain functions — easy to unit-test and shared by API and (read-only) by apps.
4	Shared contracts (<code>@bt/contracts</code>)	Zod schemas define every request and response once; web, mobile and API import the same types.
5	Side effects via an outbox	Every state change writes a <code>DomainEvent</code> in the same transaction. Notifications, points, audit and timers are produced from events — nothing is lost if a provider is down.
6	Time-based rules are data	Reminders, deadlines and flags are rows in <code>ScheduledJob</code> , executed by a job runner. Works on any host (cron, scheduled function, or <code>pg_cron</code>).
7	Offline-first mobile writes	Mobile writes go into a local outbox with an idempotency key (<code>clientRequestId</code>); the server accepts each exactly once.
8	Records that prove things are immutable	Submitted proof, plan versions, reviews, receipts, dispute evidence, ratings and audit logs cannot be edited or deleted (enforced in the API and by database triggers).

#	Principle	Why / consequence
9	Host-agnostic	Nothing depends on AWS or Azure specifics. Configuration comes from environment variables.

2.3 Technology stack

Versions are the current stable releases at the time of writing (September 2026); pin exact versions when the repository is created.

Layer	Technology	Notes
Language	TypeScript (strict)	Everywhere: web, API, mobile, shared packages.
Monorepo	pnpm workspaces + Turborepo	Shared packages, cached builds, one repo.
Web + API	Next.js (App Router), React	Server components for pages; route handlers for <code>/api/v1</code> .
UI (web)	Tailwind CSS + shadcn/ui	Fast to build, accessible components.
Mobile	React Native with Expo (development builds)	Expo Router, Camera, Location, Notifications, SQLite, FileSystem, SecureStore.
Data fetching	TanStack Query	Same caching patterns on web and mobile.
Validation	Zod	Shared request/response schemas.
ORM	Prisma 7	New <code>prisma-client</code> generator; connection URLs in <code>prisma.config.ts</code> ; PostgreSQL driver adapter.
Database	PostgreSQL (Supabase)	Also hosts <code>ScheduledJob</code> and <code>DomainEvent</code> tables.
Auth	Supabase Auth	Email + phone OTP; Send SMS Hook to plug in the chosen SMS provider.
Files	Supabase Storage	Private buckets, signed upload/download URLs.
Realtime	Supabase Realtime (Broadcast, private channels)	Chat and live project updates.
Push	Expo Notifications (FCM / APNs) + Web Push	Behind the <code>PushSender</code> interface.
Testing	Vitest, Playwright (web), Maestro (mobile)	See Section 18.
Errors / logs	Sentry (or equivalent) + structured JSON logs	Provider can change; wrapped behind a logger.

2.4 Request lifecycle

- Client sends a request to `/api/v1/...` with a Supabase access token (`Authorization: Bearer`; the web app uses the session cookie).
- Middleware verifies the JWT, loads the `User`, memberships and device trust into a **request context**.
- The handler validates input with the shared Zod schema.
- The handler calls `authorize(ctx, action, resource)` (Section 6). Denied → 403.

5. Domain logic runs in a Prisma transaction: state change + `DomainEvent` + `AuditLog` (+ `ScheduledJob` rows where needed).
6. Response is serialised with the shared schema (`BigInt` → string, dates → ISO 8601).
7. The job runner later processes the events: notifications, realtime broadcasts, point awards.

3 Codebase structure

3.1 Monorepo layout

```

buildtrack/
├─ apps/
│  └─ web/                                Next.js – web UI + REST API
│     └─ app/
│        ├── (marketing)/                public pages, contractor directory, share-link viewer
│        ├── (auth)/                    sign-in, sign-up, OTP, invitation acceptance
│        ├── client/                    Client Admin + Client Rep screens
│        ├── contractor/                Director, Site Manager, Office Coordinator screens
│        ├── admin/                     Platform Admin console
│        └─ api/v1/...                   route handlers (one folder per module)
│     └─ server/                        request context, authorize(), services, repositories
│     └─ jobs/                          job runner: event handlers + timer handlers
│  └─ mobile/                           React Native (Expo)
│     ├── app/                          Expo Router screens per role
│     ├── offline/                      SQLite store, outbox, sync engine
│     ├── capture/                      camera, GPS, compression, upload queue
│     └─ notifications/                 push registration + deep links
├─ packages/
│  ├── db/                             prisma/schema.prisma, prisma.config.ts, migrations, seed
│  ├── core/                           pure domain logic: state machines, progress, points, billing rules
│  ├── contracts/                       Zod schemas + TS types for every endpoint and event
│  ├── api-client/                      typed fetch client used by web and mobile
│  ├── integrations/                   adapter interfaces + implementations (providers TBD)
│  └─ config/                           shared eslint / tsconfig / constants (48 h pause, 60 s video, 3 days ...)
└─ supabase/                           local Supabase config, SQL for RLS, triggers, storage policies

```

3.2 Package rules

Package	May import	Must not
core	contracts, config	Import Prisma, Next.js, React or any I/O. Pure functions only.
contracts	Zod	Contain logic beyond validation.
db	Prisma	Be imported by apps/mobile.
integrations	contracts, config	Be called from UI code; only from API services / jobs.
apps/mobile	contracts, api-client, core (read-only helpers)	Talk to the database or use the Supabase service key.

3.3 Central constants

All business numbers agreed in the PRD live in one file (`packages/config/rules.ts`) so they can change without hunting through code:

```
export const RULES = {
  receiptConfirmDays: 3,           // FR-PAY-05
  receiptReminderDay: 2,           // day-2 reminder (push + email + SMS)
  reviewReminderDays: [1, 3, 5],   // FR-REV-07
  reviewOverdueDays: 7,           // FR-REV-08
  repeatedRejectionAt: 3,         // FR-REV-05
  disputeUnresolvedDays: 14,      // FR-DSP-06
  bankChangePaymentPauseHours: 48, // confirmed for SRD
  maxVideoSeconds: 60,           // confirmed for SRD
  freeProjectMaxMonths: 12,       // FR-BIL-04
  verificationValidityMonths: 12, // FR-VER-06
  defaultSiteRadiusMeters: 200,   // geofence default (editable per project)
  orgEarlyAccessActiveProjects: 3, // FR-BIL-08 (to confirm in business phase)
  stageWeightTotalBp: 10_000,     // weights must total 100 %
} as const;
```

4 Data design

The full, validated Prisma schema is in Appendix A (49 models, 45 enums) and is delivered alongside this document as `schema.prisma`.

4.1 Prisma setup

- **Prisma 7** with the `prisma-client` generator (output inside `packages/db`) and the PostgreSQL driver adapter.
- Connection strings live in `prisma.config.ts` (not in the schema): a **pooled** URL for the app and a **direct** URL for migrations.
- Migrations are created with `prisma migrate dev` and committed. Raw SQL (RLS, triggers, storage policies) lives in `supabase/` and in custom migration files.
- One shared Prisma client instance per server process.

4.2 Entity overview

Area	Models	Purpose
Identity	<code>User</code> , <code>Device</code> , <code>IdentityVerification</code>	People (id = Supabase Auth id), their trusted devices and push tokens, NIN/passport check results.
Client side	<code>ClientAccount</code> , <code>ClientMember</code> , <code>ClientRepProject</code>	Individual / organisation accounts; Client Admin and Client Reps with modules; which projects each Rep can access.
Contractor side	<code>Contractor</code> , <code>ContractorMember</code> , <code>VerificationSubmission</code> , <code>VerificationItem</code> , <code>ContractorBadge</code> , <code>BankAccount</code>	Companies, staff with fixed roles, verification checks, badges, bank accounts (history kept).
Connections	<code>Invitation</code>	Client ↔ contractor invites (incl. companies not yet registered), staff and Client Rep invites.
Presets	<code>StagePreset</code> , <code>PresetStage</code>	Platform and company presets; company default payment structure.
Projects	<code>Project</code> , <code>ProjectAssignment</code> , <code>PlanVersion</code> , <code>Stage</code> , <code>CheckListItem</code>	The project, crew assignments, immutable plan versions, live stages and checklists.
Proof & review	<code>ProgressUpdate</code> , <code>MediaAsset</code> , <code>StageReview</code> , <code>Issue</code>	Updates with GPS/time-stamped media, review rounds, issues from rejections.
Work	<code>Task</code> , <code>CheckIn</code>	Crew tasks and GPS site check-ins.
Change	<code>ChangeRequest</code>	Formal changes to a locked plan.
Money	<code>PaymentItem</code> , <code>PaymentReceipt</code>	Payment schedule (deposit, stage, change-request, retention) and uploaded receipts.
Disputes	<code>Dispute</code> , <code>DisputeEvidence</code>	Payment and stage disputes with evidence and two-sided resolution.
Communication	<code>Conversation</code> , <code>ConversationParticipant</code> , <code>Message</code> , <code>Comment</code>	Company-contained chat and in-context comments.

Area	Models	Purpose
Reputation & sharing	Rating , ShareLink	Two-way ratings; read-only project and crew viewer links.
Rewards	PointEvent , RewardPeriod , RewardPayout	Point ledger, periods and bonus records.
Billing	Plan , Subscription	Tier definitions (prices TBD) and each contractor's subscription / free project.
Platform	Notification , NotificationDelivery , NotificationPreference , DomainEvent , ScheduledJob , AuditLog , SupportTicket	Notifications and delivery tracking, event outbox, timers, audit trail, support.

4.3 Modelling rules

Rule	Detail
IDs	UUID everywhere. <code>User.id</code> equals the Supabase Auth user id, so the JWT <code>sub</code> maps straight to the user.
Money	Minor units (kobo / cents) in <code>BigInt</code> fields ending in <code>Minor</code> . Never floats. Serialised as strings in JSON. Currency is stored per project and copied onto each payment.
Weights	Basis points (<code>weightBp</code> , 10 000 = 100 %). Stage weights in a project must total exactly 10 000.
Dates	Timestamps in UTC; planning dates (<code>startDate</code> , <code>targetEnd</code>) as <code>DATE</code> . Display converts to the user's timezone.
Plan versions	Each sent / accepted plan is stored as an immutable JSON <code>snapshot</code> in <code>PlanVersion</code> . Live <code>Stage</code> and <code>PaymentItem</code> rows are (re)generated from the accepted snapshot.
Idempotency	Anything created offline carries a unique <code>clientRequestId</code> (<code>ProgressUpdate</code> , <code>CheckIn</code> , <code>Message</code>). Re-sending returns the existing row.
Media ownership	<code>MediaAsset</code> belongs to exactly one owner (update, message, receipt, evidence or review) via nullable foreign keys; a database CHECK constraint enforces "exactly one".
Identity privacy	NIN / passport numbers are never stored. <code>identityHash</code> = HMAC-SHA256(secret, normalised ID number) is unique, which enforces one Client Admin account per identity (FR-IDV-02) without keeping the number.
History over deletion	Bank accounts are superseded, not deleted; members are marked <code>REMOVED</code> ; projects are completed or cancelled, never deleted.
v1.1 readiness	<code>contractType</code> , <code>PAUSED</code> / <code>CANCELLED</code> statuses already exist; expense and materials tables are added in v1.1.

4.4 Immutability enforcement

In addition to the API never exposing edit/delete endpoints for these records, PostgreSQL triggers raise an error on `UPDATE` of protected columns or `DELETE`:

Table	Protected after
<code>PlanVersion</code>	Status leaves <code>DRAFT</code> (snapshot frozen).
<code>ProgressUpdate</code> , <code>MediaAsset</code>	<code>LockedAt</code> is set (stage submitted for review).
<code>StageReview</code>	A decision is recorded.

Table	Protected after
PaymentReceipt	Always (only status fields may move forward).
DisputeEvidence, Rating (except one response), AuditLog, PointEvent	Always.

4.5 File storage layout

Bucket (private)	Path pattern	Contents
proof	{projectId}/{stageId}/{mediaId}.{ext}	Progress photos, videos, voice notes; review media.
receipts	{projectId}/{paymentItemId}/{mediaId}.{ext}	Payment receipts.
evidence	{projectId}/{disputeId}/{mediaId}.{ext}	Dispute evidence.
chat	{contractorId}/{conversationId}/{mediaId}.{ext}	Chat attachments.
verification	{contractorId}/{submissionId}/{itemType}.{ext}	Verification documents (admin + Director only).
public	avatars/..., logos/..., portfolio/...	Non-sensitive public images.

All private files are read and written only through **signed URLs** issued by the API after a permission check (Section 9.4).

5 Authentication & sessions

5.1 Approach

- **Supabase Auth** handles sign-up, sign-in, OTP and sessions for both apps. BuildTrack's own tables hold everything else (roles, memberships, verification).
- **Web:** session in HTTP-only cookies via Supabase's server-side helpers for Next.js. **Mobile:** Supabase JS client with tokens in the device's secure storage.
- **API:** verifies the Supabase JWT on every request and builds the request context.
- **SMS OTP:** Supabase's **Send SMS Hook** routes OTP messages to the SMS provider chosen later (via the `SmsSender` adapter), so a Nigerian provider can be used even if Supabase has no built-in integration for it.

5.2 Requirements

ID	Requirement	PRD
SR-AUT-01	Sign-up collects full name, email and phone (E.164, default +234). Both are verified by OTP before the account becomes active.	FR-ACC-01/02
SR-AUT-02	On first successful sign-in the API creates the <code>User</code> row (id = auth id) and, depending on the chosen path, a <code>ClientAccount</code> + <code>ClientMember(CLIENT_ADMIN)</code> or a <code>Contractor</code> + <code>ContractorMember(DIRECTOR)</code> + <code>Subscription(FREE_PROJECT)</code> .	FR-ACC-01
SR-AUT-03	Every app install sends a stable <code>deviceKey</code> . An unknown device for that user requires an SMS OTP step-up before protected actions; on success <code>Device.trustedAt</code> is set.	FR-ACC-07
SR-AUT-04	Invited users (staff, Client Reps) sign up through the invitation link; accepting creates the membership with the invited role and marks the invitation <code>ACCEPTED</code> .	FR-ACC-03/04
SR-AUT-05	Removing a member sets <code>status = REMOVED</code> ; the API rejects their requests for that account immediately (checked per request, not cached in the JWT).	FR-ACC-06
SR-AUT-06	Client Admin identity verification (NIN or passport + selfie) is performed through the <code>IdentityVerifier</code> adapter; only the result and <code>identityHash</code> are stored. A duplicate hash is rejected.	FR-IDV-01..03
SR-AUT-07	Platform Admins are flagged with <code>User.isPlatformAdmin</code> , must use a trusted device, and every admin action is written to <code>AuditLog</code> .	FR-ADM-06
SR-AUT-08	Auth endpoints are rate-limited per phone, email and IP (OTP requests in particular).	—

5.3 Request context

```
type RequestContext = {
  user: { id: string; isPlatformAdmin: boolean; timezone: string };
  device: { id: string; trusted: boolean };
  clientMemberships: {
    clientAccountId: string; memberId: string;
    role: 'CLIENT_ADMIN' | 'CLIENT_REP'; modules: ClientRepModule[];
    projectIds: string[] /* Reps only */ []];
  contractorMemberships: {
    contractorId: string; memberId: string;
    role: ContractorRole; assignedProjectIds: string[] /* crew */ []];
  shareToken?: { projectId: string; kind: 'PROJECT_SHARE' | 'CREW_VIEWER' };
};
```

6 Authorisation

6.1 Model

Permissions are decided in code by one function, called by every handler:

```
authorize(ctx: RequestContext, action: Action, resource: { projectId?: contractorId?: clientId?: ... })
  → { allowed: true } | { allowed: false; reason: string }
```

It resolves the user's **side** and **role** for the resource, then applies the rule for the action. Rules are a table in `@bt/core` that mirrors PRD Section 5.3, so the matrix and the code cannot drift apart (a unit test checks every row).

6.2 Scoping rules

Who	Can reach a project when...
Client Admin	<code>project.clientAccountId</code> is their client account.
Client Rep	Their client account owns the project and a <code>ClientRepProject</code> row exists and the action's module is in <code>modules</code> .
Director, Site Manager, Office Coordinator	<code>project.contractorId</code> is their company (all company projects).
Crew Member	Their company owns the project and a <code>ProjectAssignment</code> exists.
Share-link viewer	A valid, unrevoked, unexpired token for that project; read-only; never payments, bank details or chat.
Crew viewer link	Like a share link, but requires a signed-in Crew Member of the same company .
Platform Admin	Administrative endpoints only (<code>/api/v1/admin/*</code>); every access audited.

6.3 Action catalogue (excerpt)

Full list lives in code; key actions and their rules (C.Admin = Client Admin, Rep(M) = Client Rep with module M):

Action	Allowed for
<code>project.create</code> , <code>plan.edit</code>	Director, Site Manager, Office Coordinator
<code>plan.send</code> , <code>client.invite</code>	Director, Site Manager
<code>plan.accept</code> , <code>plan.requestChanges</code>	C.Admin only
<code>changeRequest.raise</code> / <code>.decide</code>	C.Admin, Rep(CHANGE_REQUESTS), Director, Site Manager
<code>update.create</code>	Director, Site Manager, Office Coordinator, assigned Crew
<code>stage.submit</code>	Director, Site Manager
<code>stage.review</code> (approve / reject)	C.Admin, Rep(REVIEW_STAGES)
<code>payment.viewAccount</code> , <code>receipt.upload</code> / <code>.withdraw</code>	C.Admin, Rep(PAYMENTS)
<code>receipt.confirm</code> / <code>.dispute</code>	Director, Office Coordinator

Action	Allowed for
dispute.open / .addEvidence / .markResolved	C.Admin, Rep(PAYMENTS or REVIEW_STAGES by type), Director; Site Manager (stage), Office Coordinator (payment)
bank.change, company.manage, subscription.manage	Director only
staff.invite	Director (any role); Site Manager (Crew only)
task.manage, crew.assign	Director, Site Manager
chat.project	C.Admin, Rep(MESSAGE_CONTRACTOR), Director, Site Manager, Office Coordinator
chat.internal	All contractor roles
shareLink.manage	C.Admin, Rep(SHARE_LINKS), Director, Site Manager, Office Coordinator
crewViewerLink.create	Crew Member (for their assigned projects)
rating.create	C.Admin (rates contractor), Director (rates client)
rewards.configure, rewards.markPaid	Director
rewards.recognise	Director (anyone), Site Manager (Crew only)
clientRep.manage	C.Admin; Rep(INVITE_PEOPLE) may invite Reps

6.4 Database-level protection

- **RLS is enabled on every application table** with no policies for the `anon` and `authenticated` roles, so even a leaked anon key cannot read data through Supabase's auto-generated APIs. Prisma uses a privileged connection and is unaffected.
- **Realtime:** RLS policies on `realtime.messages` allow joining a private topic only if the user belongs to it (Section 12.2).
- **Storage:** buckets are private; no direct policies for clients — access only by API-issued signed URLs.

7 API design

One REST API under `/api/v1`, served by Next.js route handlers and used identically by the web app, the mobile app and the admin console.

7.1 Conventions

Topic	Rule
Style	Resource-oriented REST with explicit action endpoints for state changes (e.g. <code>POST /stages/{id}/submit</code>) instead of generic PATCHes, so every transition is validated and audited.
Format	JSON. <code>BigInt</code> money as strings; timestamps ISO 8601 UTC; enums as upper-case strings.
Validation	Every body, query and response has a Zod schema in <code>@bt/contracts</code> ; invalid input → <code>400 VALIDATION_FAILED</code> with field errors.
Auth	Authorization: Bearer <code><supabase access token></code> (mobile) or session cookie (web). Public endpoints are marked.
Idempotency	Create endpoints used offline accept a <code>clientRequestId</code> . Other POSTs accept an optional <code>Idempotency-Key</code> header.
Pagination	Cursor-based: <code>?cursor=&limit=</code> (default 30, max 100); responses include <code>nextCursor</code> .
Concurrency	State-changing actions check the current state inside the transaction; a stale action returns <code>409 INVALID_STATE</code> with the current state.
Errors	<code>{ "error": { "code": "...", "message": "...", "details": {...} } }</code> — codes in Appendix D.
Versioning	<code>/v1</code> prefix. Breaking changes go to <code>/v2</code> ; the mobile app sends <code>X-App-Version</code> so old builds can be told to update.
Rate limits	Per user and per IP; stricter on auth, OTP, invitations and uploads.

7.2 Endpoint catalogue

Paths are relative to `/api/v1`. "Scoped" means any user who can reach the project under Section 6.2 with the relevant module. Abbreviations: C.Admin = Client Admin, Rep(M) = Client Rep with module M, SM = Site Manager, OC = Office Coordinator.

Auth, me & devices

Method	Path	Who	Purpose	PRD
POST	<code>/auth/bootstrap</code>	Signed-in user	Create User + account after first OTP sign-in (client or contractor path)	FR-ACC-01
GET	<code>/me</code>	Any	Profile, memberships, roles, modules, unread counts	
PATCH	<code>/me</code>	Any	Name, avatar, timezone, quiet hours	FR-NTF-03
POST	<code>/me/devices</code>	Any	Register device + push token; returns trust status	FR-ACC-07
POST	<code>/me/devices/{id}/trust</code>	Any	Confirm new device with SMS OTP	FR-ACC-07
POST	<code>/me/identity</code>	Client Admin	Start NIN / passport + selfie verification	FR-IDV-01

Method	Path	Who	Purpose	PRD
GET	/invitations/{token}	Public	Preview an invitation	
POST	/invitations/{token}/accept	Signed-in user	Accept staff / Client Rep / project invitation	FR-DIR-03 . . 05

Client accounts & Client Reps

Method	Path	Who	Purpose	PRD
GET	/clients/{id}	C.Admin, Rep	Account overview	
GET	/clients/{id}/reps	C.Admin	List Client Reps and modules	FR-ACC-03
POST	/clients/{id}/reps	C.Admin, Rep(INVITE)	Invite a Client Rep with modules and projects	FR-ACC-03
PATCH	/clients/{id}/reps/{memberId}	C.Admin	Change modules / project access; remove	FR-ACC-03/06

Contractors, staff & verification

Method	Path	Who	Purpose	PRD
GET	/contractors/{id}	Members	Company profile, badges, subscription summary	
PATCH	/contractors/{id}	Director	Edit profile, project types, reward settings	FR-RWD-01
GET	/contractors/{id}/members	Director, SM, OC	Staff list with roles	
POST	/contractors/{id}/members	Director, SM (crew)	Invite staff with fixed role	FR-ACC-04
PATCH	/contractors/{id}/members/{memberId}	Director	Change role, work days; remove	FR-ACC-06
POST	/contractors/{id}/verification	Director	Create submission; returns upload URLs per item	FR-VER-01
POST	/contractors/{id}/verification/{sid}/submit	Director	Submit; runs automated CAC / TIN / ID / bank checks	FR-VER-02/03
GET	/contractors/{id}/verification	Director	Status and per-item results	FR-VER-01
POST	/contractors/{id}/bank-accounts	Director	Add / replace account: name lookup, CAC match, 48 h pause, alerts	FR-BNK-01 . . 04

Directory & invitations

Method	Path	Who	Purpose	PRD
GET	/directory	Public / Clients	Search verified contractors (location, type, badge, rating)	FR-DIR-01
GET	/directory/{contractorId}	Public / Clients	Public profile, ratings, unresolved-dispute flags	FR-DIR-02
POST	/invitations	C.Admin, Rep(INVITE), Director, SM	Client→contractor (listed or new) or contractor→client invite	FR-DIR-03 . . 05
POST	/invitations/{id}/respond	Invitee	Accept / decline	FR-DIR-03

Presets

Method	Path	Who	Purpose	PRD
GET	/presets	Contractor staff	Platform + company presets (filter by project type)	FR-PRE-01
POST	/presets	Director, SM	Copy platform preset into a company preset	FR-PRE-03
PATCH	/presets/{id}	Director, SM	Edit stages, weights, checklist, proof rules, default payment structure	FR-PRE-03/04

Projects & plans

Method	Path	Who	Purpose	PRD
GET	/projects	Any member	Projects visible to caller, with summary (verified %, reported %, ball in court)	\$6.2 PRD
POST	/projects	Director, SM, OC	Create draft from preset (copies stages)	FR-PRJ-01..03
GET	/projects/{id}	Scoped	Full project detail	
PATCH	/projects/{id}	Director, SM, OC	Edit draft fields (not after lock)	FR-PRJ-01
GET	/projects/{id}/plan	Scoped	Current plan + version history	FR-PLN-06
PUT	/projects/{id}/plan/draft	Director, SM, OC	Save draft plan: stages, dates, weights, payment structure	FR-PLN-01..04
POST	/projects/{id}/plan/send	Director, SM	Freeze draft as PlanVersion(SENT); notify client	FR-PLN-05
POST	/projects/{id}/plan/accept	C.Admin	Accept → lock; generate Stages + PaymentItems; project ACTIVE	FR-PLN-06
POST	/projects/{id}/plan/request-changes	C.Admin	Return with comments	FR-PLN-05
POST	/projects/{id}/assignments	Director, SM	Assign / unassign crew	FR-ACC-05
POST	/projects/{id}/complete	Director	Mark complete when all stages approved and payments settled	FR-PRJ-05

Change requests

Method	Path	Who	Purpose	PRD
GET	/projects/{id}/change-requests	Scoped	List	
POST	/projects/{id}/change-requests	C.Admin, Rep(CR), Director, SM	Raise with structured changes + impact	FR-CHG-01/02
POST	/change-requests/{id}/decide	Other side	Approve → new PlanVersion, regenerate affected rows; or reject	FR-CHG-03/04

Stages, proof & review

Method	Path	Who	Purpose	PRD
GET	/projects/{id}/stages	Scoped	Stages with status, proof counts, flags	
PATCH	/stages/{id}/progress	Director, SM	Set reported % (contractor estimate)	FR-PCT-02
PATCH	/checklist-items/{id}	Staff, assigned crew	Mark done / undone (before submit)	FR-PRF-01

Method	Path	Who	Purpose	PRD
POST	/media/upload-urls	Uploader	Request signed upload URLs for media about to be sent	FR-PRF-02
POST	/projects/{id}/updates	Staff, assigned crew	Create update (idempotent via clientRequestId) with media metadata	FR-PRF-01..05
GET	/projects/{id}/updates	Scoped	Timeline; filter by stage / checklist item / date	FR-PRF-08
POST	/stages/{id}/submit	Director, SM	Validate minimum proof + checklist → READY_FOR_REVIEW; lock proof	FR-PRF-04, FR
POST	/stages/{id}/review	C.Admin, Rep(REVIEW)	Approve, or reject with failed items + comments + media	FR-REV-02/03
PATCH	/issues/{id}	Director, SM	Mark fixed (with proof)	FR-REV-04

Tasks & check-ins

Method	Path	Who	Purpose	PRD
GET	/projects/{id}/tasks	Staff, assigned crew	Tasks (crew: own tasks)	
POST	/projects/{id}/tasks	Director, SM	Create and assign	FR-TSK-01
POST	/tasks/{id}/complete	Assigned crew	Mark done with proof	FR-TSK-02
POST	/tasks/{id}/confirm	Director, SM	Confirm completion (awards points)	FR-TSK-02
POST	/projects/{id}/check-ins	Staff, assigned crew	GPS check-in (idempotent); geofence evaluated server-side	FR-TSK-03

Payments

Method	Path	Who	Purpose	PRD
GET	/projects/{id}/payments	C.Admin, Rep(PAY), Director, OC	Schedule with statuses; bank details only when DUE and not paused	FR-PAY-02/03
POST	/payments/{id}/receipts	C.Admin, Rep(PAY)	Upload receipt: transaction ref, amount, date, file; starts 3-day clock	FR-PAY-04
POST	/receipts/{id}/withdraw	C.Admin, Rep(PAY)	Withdraw before confirmation	FR-PAY-07
POST	/receipts/{id}/confirm	Director, OC	Confirm receipt	FR-PAY-05
POST	/receipts/{id}/dispute	Director, OC	Dispute with reason → opens payment dispute	FR-PAY-05

Disputes

Method	Path	Who	Purpose	PRD
GET	/projects/{id}/disputes	Scoped (per type)	List with status and 14-day countdown	
POST	/projects/{id}/disputes	Per 6.3	Open payment or stage dispute (reason required); project on hold	FR-DSP-01..03
POST	/disputes/{id}/evidence	Both sides	Add note + media (immutable)	FR-DSP-02
POST	/disputes/{id}/mark-resolved	Both sides	Record this side's resolution; closes when both sides have	FR-DSP-05

Chat & comments

Method	Path	Who	Purpose	PRD
GET	/conversations	Participants	Conversations with unread counts	FR-CHT-01/02
POST	/conversations	Contractor staff	Create direct / group chat inside the company	FR-CHT-01
GET	/conversations/{id}/messages	Participants	Paginated history (cursor)	FR-CHT-05
POST	/conversations/{id}/messages	Participants	Send (idempotent); broadcast via Realtime	FR-CHT-03
POST	/conversations/{id}/read	Participants	Update lastReadAt	FR-CHT-06
GET	/comments?targetType&targetId	Scoped	Comments on a stage / update / issue / payment / change request	FR-CHT-04
POST	/comments	Scoped	Add comment	FR-CHT-04

Notifications

Method	Path	Who	Purpose	PRD
GET	/notifications	Any	In-app notification list	FR-NTF-01
POST	/notifications/read	Any	Mark read	
GET	/notification-preferences	Any	Per-event channel settings	FR-NTF-03
PUT	/notification-preferences	Any	Update (critical events cannot be disabled)	FR-NTF-04

Ratings & share links

Method	Path	Who	Purpose	PRD
POST	/projects/{id}/ratings	C.Admin, Director	Rate the other side after completion	FR-RAT-01
POST	/ratings/{id}/response	Rated side	One public response	FR-RAT-04
POST	/projects/{id}/share-links	Per 6.3	Create project share link or crew viewer link	FR-SHR-01/04
DELETE	/share-links/{id}	Creator / admins of project	Revoke	FR-SHR-03
GET	/shared/{token}	Public (token)	Read-only project view (no payments, bank, chat)	FR-SHR-02

Rewards

Method	Path	Who	Purpose	PRD
GET	/contractors/{id}/rewards/me	SM, OC, Crew	Own points, history, value	FR-RWD-05
GET	/contractors/{id}/rewards	Director, SM	Everyone's points; leaderboard if enabled	FR-RWD-05/06
POST	/contractors/{id}/rewards/recognition	Director, SM (crew)	Give capped recognition points with reason	FR-RWD-07
POST	/contractors/{id}/rewards/periods/{pid}/close	Director	Close period, compute payouts	FR-RWD-04
POST	/reward-payouts/{id}/mark-paid	Director	Record bonus paid	FR-RWD-04

Billing

Method	Path	Who	Purpose	PRD
GET	/contractors/{id}/subscription	Director	Plan, active-project usage, free-project status	FR-BIL-01..04
POST	/contractors/{id}/subscription/checkout	Director	Start plan purchase via BillingProvider (provider TBD)	FR-BIL-05
POST	/webhooks/billing	Provider (signed)	Payment result → subscription status	FR-BIL-07

Sync (mobile)

Method	Path	Who	Purpose	PRD
POST	/sync/push	Mobile	Batch of outbox operations (idempotent); per-item results	FR-PRF-05
GET	/sync/pull?since=cursor	Mobile	Changes since cursor for the user's projects	

Platform Admin

Method	Path	Who	Purpose	PRD
GET	/admin/verifications	Platform Admin	Verification queue	FR-ADM-01
POST	/admin/verifications/{sid}/decide	Platform Admin	Approve / reject with reason; grant badges	FR-ADM-01
GET	/admin/users	Platform Admin	Search users / accounts	FR-ADM-02
POST	/admin/accounts/{id}/suspend	Platform Admin	Suspend / reinstate	FR-ADM-02
GET	/admin/risk	Platform Admin	Unconfirmed payments, unresolved disputes, repeated rejections, bank changes	FR-ADM-03
GET	/admin/subscriptions	Platform Admin	Subscriptions, free projects, Early Access orgs	FR-ADM-04
POST	/admin/presets	Platform Admin	Create / edit platform presets	FR-ADM-05
GET	/admin/audit	Platform Admin	Audit log search	FR-ADM-06
POST	/admin/tickets/{id}	Platform Admin	Support ticket handling	FR-ADM-02

Internal

Method	Path	Who	Purpose	PRD
POST	/internal/jobs/tick	Scheduler (secret)	Process due ScheduledJobs and pending DomainEvents	
POST	/internal/auth/send-sms	Supabase Send SMS Hook (signed)	Deliver OTP through SmsSender adapter	FR-ACC-02

8 Domain logic & state machines

All transitions below are implemented as pure functions in `@bt/core` (`next = transition(current, event, facts)`) and executed by API services inside one database transaction together with their `DomainEvent`, `AuditLog` and `ScheduledJob` rows.

8.1 Plan lifecycle

From	Event	To	Guards & side effects
DRAFT	send	SENT	Weights total 10 000; every stage has dates, checklist, proof rules; payment items sum $\leq$ contract value. Snapshot frozen. Project $\rightarrow$ <code>AWAITING_CLIENT_ACCEPTANCE</code> . Event <code>PLAN_SENT</code> .
SENT	requestChanges	CHANGES_REQUESTED	Comment required. Contractor edits a new DRAFT version. Event <code>PLAN_CHANGES_REQUESTED</code> .
SENT	accept	ACCEPTED	C.Admin only. Generates <code>Stage</code> , <code>ChecklistItem</code> and <code>PaymentItem</code> rows from the snapshot; project $\rightarrow$ <code>ACTIVE</code> ; deposit (if any) $\rightarrow$ <code>DUE</code> . Event <code>PLAN_ACCEPTED</code> .
ACCEPTED	changeApproved	SUPERSEDED	A new ACCEPTED version is created from the change request; live rows updated without touching approved stages or confirmed payments.

8.2 Stage lifecycle

From	Event	To	Guards & side effects
NOT_STARTED	first update / start	IN_PROGRESS	Project must be ACTIVE and not on hold.
IN_PROGRESS	submit	READY_FOR_REVIEW	All checklist items done; counted media $\geq$ <code>minPhotos / minVideos</code> (only <code>capturedInApp = true</code> and uploaded); description if required; project not on hold. Creates <code>StageReview(round n)</code> ; sets <code>lockedAt</code> on its updates and media. Schedules reminder jobs (day 1, 3, 5) and overdue job (day 7). Event <code>STAGE_READY_FOR_REVIEW</code> .
READY_FOR_REVIEW	approve	APPROVED	Reviewer = C.Admin or Rep(<code>REVIEW_STAGES</code>). Cancels pending reminder jobs; clears <code>reviewOverdue</code> ; linked stage payment $\rightarrow$ <code>DUE</code> ; recalculates verified %; awards points. Event <code>STAGE_APPROVED</code> .
READY_FOR_REVIEW	reject	IN_PROGRESS	At least one failed item or comment required. Creates one <code>Issue</code> per failed item; <code>rejectionCount++</code> ; if count reaches 3 $\rightarrow$ <code>repeatedRejection = true</code> (event <code>STAGE_REPEATED_REJECTION</code>). Cancels reminder jobs. Event <code>STAGE_REJECTED</code> .

From	Event	To	Guards & side effects
any open state	dispute opened	(unchanged)	Project <code>onHold = true</code> ; no submissions until resolved.

Flags (not states): `reviewOverdue` set by the day-7 job — the contractor may start the next stage; `delayed` set daily when `today > targetEnd` and not approved.

8.3 Payment lifecycle

From	Event	To	Guards & side effects
<code>SCHEDULED</code>	trigger met	<code>DUE</code>	Deposit: plan accepted. Stage: stage approved. Change request: CR approved. Retention release: project handover. Event <code>PAYMENT_DUE</code> .
<code>DUE</code>	receipt uploaded	<code>RECEIPT_UPLOADED</code>	Transaction ref, amount, date, file required. Records which <code>BankAccount</code> was shown. <code>confirmDeadline = now + 3 days</code> ; schedules day-2 reminder and deadline job. Event <code>RECEIPT_UPLOADED</code> .
<code>RECEIPT_UPLOADED</code>	confirm	<code>CONFIRMED</code>	Director / OC. Cancels deadline jobs; awards OC points if on time. Event <code>PAYMENT_CONFIRMED</code> .
<code>RECEIPT_UPLOADED</code>	dispute	<code>DISPUTED</code>	Reason required; opens a <code>Dispute(PAYMENT)</code> ; project on hold.
<code>RECEIPT_UPLOADED</code>	deadline passed	<code>UNCONFIRMED</code>	"Paid – not confirmed by contractor"; flagged in admin risk view. Contractor can still confirm or dispute later.
<code>RECEIPT_UPLOADED</code>	withdraw	<code>DUE</code>	Client withdraws before confirmation; receipt → <code>WITHDRAWN</code> (kept).
<code>DISPUTED</code>	dispute resolved	<code>CONFIRMED / DUE</code>	Confirmed if the contractor confirmed; back to <code>DUE</code> if the client withdrew the receipt.

Payment pause: while `Contractor.paymentsPausedUntil > now` (48 h after a bank-detail change), bank details are hidden and receipt upload is blocked (423 `PAYMENTS_PAUSED`).

8.4 Dispute lifecycle

Event	Effect
open	Reason required; <code>status = OPEN</code> ; project <code>onHold = true</code> ; schedule 14-day job. Event <code>DISPUTE_OPENED</code> .
addEvidence	Immutable <code>DisputeEvidence</code> with media.
markResolved (client)	Sets <code>clientResolvedAt</code> .
markResolved (contractor)	Sets <code>contractorResolvedAt</code> .
both timestamps set	<code>status = RESOLVED</code> ; <code>resolvedAt</code> ; project <code>onHold = false</code> if no other open dispute; underlying payment / stage updated. Event <code>DISPUTE_RESOLVED</code> .

Event	Effect
14 days, still open	unresolvedFlaggedAt set → "Unresolved dispute" shown on project, contractor profile and client profile. Event DISPUTE_UNRESOLVED (push + email + SMS).

8.5 Change request lifecycle

PENDING → APPROVED (other side; new plan version; affected stages / payments regenerated; approved stages and confirmed payments untouched) or REJECTED, or WITHDRAWN by the raiser. Approved extra work may add a PaymentItem(CHANGE_REQUEST) which becomes DUE on approval (FR-CHG-04).

8.6 Project lifecycle

DRAFT → AWAITING_CLIENT_ACCEPTANCE → ACTIVE → COMPLETED. CLIENT_NOT_CONNECTED is used when a contractor activates a project without a client (FR-NOC-01): stages can progress, but stage.review is impossible and only reported progress shows. When a client joins and accepts the plan, the project becomes ACTIVE. PAUSED and CANCELLED are reserved for v1.1. onHold is a separate flag controlled by disputes.

Complete requires every stage APPROVED and every payment CONFIRMED (retention release included). Completion opens ratings and, for a free project, ends the free period.

8.7 Progress calculation

```

verifiedPercent = sum(stage.weightBp where status = APPROVED) / 100           // 0–100, 2 dp
reportedPercent = sum(stage.weightBp * stage.reportedPercent / 100) / 100    // contractor estimate
paidMinor       = sum(payment.amountMinor where status = CONFIRMED)
remainingMinor  = contractValueMinor + approvedChangeRequests - paidMinor
ballInCourt     = first of: open dispute → both sides; READY_FOR_REVIEW stage → client;
                  RECEIPT_UPLOADED payment → contractor; DUE payment → client;
                  PENDING change request → other side; otherwise → contractor

```

Stages reviewOverdue are excluded from verified % until approved (FR-REV-09). Values are computed on read (cheap) and cached per project for dashboards.

8.8 Timers (ScheduledJob)

Job type	Scheduled when	Action when it runs
REVIEW_REMINDER	Stage submitted: +1, +3, +5 days	If still READY_FOR_REVIEW, notify reviewers (day 5 adds SMS).
REVIEW_OVERDUE	Stage submitted: +7 days	Set reviewOverdue ; notify both sides.
RECEIPT_REMINDER	Receipt uploaded: +2 days	If unconfirmed, notify Director / OC (push + email + SMS).
RECEIPT_DEADLINE	Receipt uploaded: +3 days	If unconfirmed → payment UNCONFIRMED ; notify client; admin risk flag.
DISPUTE_UNRESOLVED	Dispute opened: +14 days	If still open → set flag on project and both profiles; notify.
PAYMENT_PAUSE_END	Bank change: +48 hours	Clear pause; notify clients with DUE payments.
STAGE_DELAY_SWEEP	Daily	Set delayed on stages past target end.

Job type	Scheduled when	Action when it runs
FREE_PROJECT_END	Free project created: +12 months	If not finished: project becomes read-only for the contractor until a plan is purchased (client keeps full view).
VERIFICATION_EXPIRY	Verified: +12 months (reminders at -30 / -7 days)	Set EXPIRED ; block new projects until re-verified.
REWARD_PERIOD_CLOSE	Per contractor config (period TBD)	Compute payouts for the period.
NOTIFICATION_DIGEST	Daily at user's local 18:00	Send progress-update digest to users who chose it.

Each job has a unique `dedupeKey` (e.g. `REVIEW_REMINDER:{stageReviewId}:3`) so rescheduling never duplicates. Handlers re-check state before acting, so a cancelled condition simply ends the job. The runner is triggered by `POST /internal/jobs/tick` every minute from any scheduler (cron, scheduled function or `pg_cron`) — the host is decided later.

8.9 Domain events

Events are written to `DomainEvent` in the same transaction as the change and processed by the runner. Each handler is idempotent. Consumers:

- **Notifications** — map event → recipients → channels (Section 13, Appendix C).
- **Realtime** — broadcast a lightweight "project changed" signal so open screens refresh.
- **Rewards** — award points for qualifying events (Section 15).
- **Search / caches** — refresh project summaries and directory stats.

9 Proof & media pipeline

9.1 Capture (mobile)

- In-app camera for photos and video; microphone for voice notes. Each capture records: device time, GPS latitude/longitude/accuracy (if permission given), uploader, and `capturedInApp = true`.
- Gallery picks are allowed but saved with `capturedInApp = false`; the UI labels them **"Not captured in app"** and they **do not count** toward minimum proof.
- Video recording stops automatically at **60 seconds**; longer gallery videos are rejected client-side and server-side.

9.2 Processing

Type	On-device processing before upload
Photo	Resize to max 1920 px long edge, JPEG ~0.75 quality, strip EXIF except orientation; GPS/time kept in metadata sent to the API (not in the file).
Video	Transcode to 720p H.264, ≤ 60 s; target ≤ ~15 MB.
Voice	AAC mono; ≤ 5 minutes.
Document	PDF / image receipts ≤ 10 MB.

A SHA-256 hash is computed on device and re-checked on the server; it is stored on `MediaAsset.sha256` as tamper evidence.

9.3 Geofence

Each project has a pin and `siteRadiusM` (default 200 m, editable by the contractor). The server computes `withinSite` from the capture GPS (distance ≤ radius + accuracy). Media captured outside the site is **flagged** to the reviewer (FR-PRF-03); check-ins outside the site do not count (FR-TSK-03). No GPS → `withinSite = null`, shown as "No location".

9.4 Upload flow

1. App asks `POST /media/upload-urls` with file metadata (kind, size, duration, hash, capture data, owner type). API checks permission and limits, creates `MediaAsset (PENDING)` rows and returns signed upload URLs (short expiry).
2. App uploads files directly to Supabase Storage (resumable for video), in the background, retrying on failure.
3. App calls the owning endpoint (e.g. `POST /projects/{id}/updates`) with the media ids; the API verifies each object exists and its size/hash match, then marks `UPLOADED`.
4. Viewing uses short-lived signed download URLs (thumbnails cached on device).

9.5 Minimum-proof check

```
countedPhotos = media where kind=PHOTO and capturedInApp and uploadStatus=UPLOADED and update.stageId=stage.id
countedVideos = same for VIDEO
canSubmit = checklist.every(done) && countedPhotos ≥ stage.minPhotos && countedVideos ≥ stage.minVideos
            && (!stage.requireDescription || some update has a description)
```

10 Mobile app (React Native / Expo)

10.1 Foundations

- Expo with **development builds** (not Expo Go), Expo Router for navigation, TanStack Query for server data, Supabase JS for auth only.
- Native modules: camera, location, notifications, SQLite, file system, secure storage, background upload / task.
- One app for everyone; after sign-in the app loads `/me` and shows the navigation for the user's role. A user with several memberships (e.g. a Client Admin who is also a Director) can switch account from a menu.
- Android first in testing: the app must stay fast on low-cost Android phones (small bundle, lazy screens, image thumbnails).

10.2 Navigation by role

Role	Tabs / main screens
Client Admin / Rep	Projects (list + verified %) · Waiting for you · Chat · Notifications · Account (Client Reps, identity, share links)
Director	Projects · Approvals & payments · Chat · Team (staff, rewards) · Company (verification, bank, subscription)
Site Manager	Today (tasks, crew, stages) · Projects · Chat · Issues
Office Coordinator	Receipts to confirm · Projects · Chat · Documents
Crew Member	One screen: today's tasks, big camera button, check-in, points. Chat as a secondary tab.

10.3 Offline architecture

Part	Design
Local store	SQLite database holding the user's projects, stages, checklist, tasks, recent updates and chat (read cache), plus an outbox table.
Outbox	Every offline-capable write (update, checklist tick, task completion, check-in, chat message) is saved locally with a UUID <code>clientRequestId</code> , its payload and media file paths, then shown immediately in the UI as "Waiting to sync".
Sync push	When online, media are uploaded first (Section 9.4), then operations are sent in order via <code>POST /sync/push</code> . The server applies each once (idempotent) and returns per-item results; failures are shown to the user with a reason (e.g. "stage already submitted").
Sync pull	<code>GET /sync/pull?since=cursor</code> returns changes for the user's projects; also triggered by realtime "project changed" signals and on app foreground.
Time & place	Capture time and GPS are those of the original capture, never the sync time (FR-PRF-05). Server stores both <code>capturedAt</code> and <code>createdAt</code> .
Conflicts	Server state wins for transitions (submit, approve, confirm); these actions require a connection. Additive data (updates, media, messages, check-ins) never conflicts.
Online-only actions	Stage submit and review, plan accept, receipt upload / confirm, disputes, bank changes, billing.

10.4 Push notifications & deep links

- On sign-in the app registers its push token via `POST /me/devices`.
- Every notification carries a `deepLink` (e.g. `buildtrack://projects/{id}/stages/{stageId}`) that opens the exact item (FR-NTF-05).
- Tokens that fail delivery repeatedly are removed.

11 Web app (Next.js)

11.1 Route map

Route group	Pages
(marketing)	Home, contractor directory and profiles (public, server-rendered for search engines), share-link viewer <code>/s/[token]</code> .
(auth)	Sign in, sign up (client / contractor path), OTP, invitation acceptance, new-device verification.
<code>/client</code>	Dashboard (all projects), project detail (stages, proof gallery, timeline, payments, disputes, change requests, chat), Client Reps, identity verification, share links, notifications, settings.
<code>/contractor</code>	Dashboard, projects, project setup wizard (preset → plan → send), stages and review status, tasks and crew, payments and receipts, disputes, presets, team, rewards, verification, bank account, subscription, chat.
<code>/admin</code>	Verification queue, users and accounts, risk view, subscriptions, preset library, audit log, support tickets.

11.2 Rendering rules

- Pages are server components that call the same service layer the API uses (no internal HTTP hop); interactive parts are client components calling `/api/v1` through `@bt/api-client` .
- Directory and share-link pages are server-rendered and cacheable; everything signed-in is dynamic.
- Web push via the browser Push API for users who allow it.
- Responsive down to phone width so clients can use the web app on mobile browsers too.

12 Chat & realtime

12.1 Design

- Every conversation belongs to exactly one contractor company (`Conversation.contractorId`), which keeps chat "contained within the company and its clients" (FR-CHT).
- Types: `COMPANY_DIRECT` and `COMPANY_GROUP` (staff only) and `PROJECT` (the project's client side + contractor staff, created automatically when a client connects).
- **Writes go through the API** (`POST /conversations/{id}/messages` , idempotent), which saves the message, then publishes it on the conversation's Realtime channel.
- **Reads:** clients subscribe to private Realtime channels for live delivery and use the API for history.
- Participants who leave (removed staff) keep no access; their past messages stay (FR-CHT-05).
- Crew Members see company chats they are added to; they have no project chat with the client by default (PRD 5.3).

12.2 Channel security

Channel topic	Who may join (RLS policy on <code>realtime.messages</code>)
<code>conv:{conversationId}</code>	Users with an active <code>ConversationParticipant</code> row.
<code>project:{projectId}</code>	Users who can reach the project (Section 6.2); used for "project changed" signals only — no sensitive data in the payload.
<code>user:{userId}</code>	That user only; new notifications, unread counts.

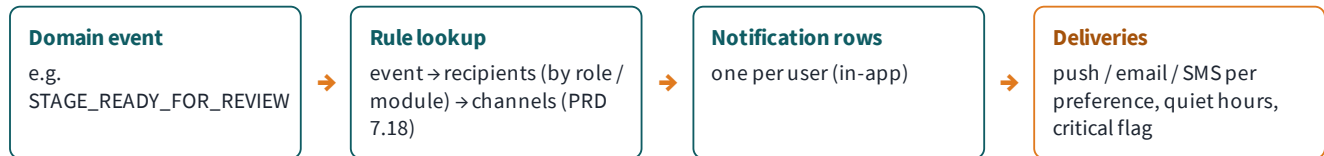
Channels are created with `private: true`; policies use `realtime.topic()` and the JWT `sub` to check membership. Because policies are cached for the connection, removing a participant also forces a token refresh / disconnect for that user.

12.3 Features

Text, photos, videos (≤ 60 s), documents, voice notes; delivery and read state from `lastReadAt` ; offline sending through the outbox; unread counts per conversation; in-context comments via the `Comment` model on stages, updates, issues, payments and change requests.

13 Notifications engine

13.1 Pipeline



13.2 Rules

ID	Requirement
SR-NTF-01	Every notification is stored in-app (<code>Notification</code>) regardless of other channels.
SR-NTF-02	Channel defaults come from the PRD tables (Appendix C lists each event's code, recipients and channels). Push is primary ; email for action/record events; SMS only for critical ones. WhatsApp is not used.
SR-NTF-03	Client Rep recipients are filtered by module (e.g. payment events only to Reps with <code>PAYMENTS</code>).
SR-NTF-04	User preferences (<code>NotificationPreference</code>) can switch push/email/SMS per event; critical events (OTP, new device, bank-detail change, disputes) ignore preferences and quiet hours.
SR-NTF-05	Quiet hours (user local time) delay non-critical push and SMS until the window ends; email is not delayed.
SR-NTF-06	Progress-update notifications can be batched into a daily digest (<code>NOTIFICATION_DIGEST</code> job).
SR-NTF-07	Each delivery retries with back-off (max 5 attempts). For critical events, if push fails or the user has no push token, email is sent; if the event is SMS-eligible, SMS is sent as the last resort.
SR-NTF-08	Every message has a deep link; emails contain a button to the same item on the web app.
SR-NTF-09	Templates live in code (<code>packages/contracts/notifications</code>) with plain, short English copy.

14 Integrations (adapter interfaces)

Providers are not chosen yet. Each external capability is a TypeScript interface in `packages/integrations`. A **fake** implementation (logs to console, deterministic results) is used in development and tests; real providers are added later without touching business code.

```
interface BusinessVerifier { // CAC + TIN (FR-VER-02)
  lookupCac(rcNumber: string): Promise<{ name: string; status: 'ACTIVE'|'INACTIVE'|'UNKNOWN';
    incorporatedOn?: string; raw: unknown }>;
  lookupTin(tin: string): Promise<{ valid: boolean; name?: string; raw: unknown }>;
}
interface IdentityVerifier { // NIN / passport + selfie (FR-VER director ID, FR-IDV-01)
  startCheck(input: { method: 'NIN'|'PASSPORT'; idNumber: string; selfieUrl: string;
    fullName: string; dateOfBirth?: string }): Promise<{ ref: string }>;
  getResult(ref: string): Promise<{ status: 'PENDING'|'PASSED'|'FAILED'; reason?: string }>;
}
interface BankAccountResolver { // name lookup (FR-VER-03, FR-BNK-02)
  listBanks(): Promise<{ code: string; name: string }[]>;
  resolve(bankCode: string, accountNumber: string): Promise<{ accountName: string }>;
}
interface SmsSender { send(to: string, text: string): Promise<{ id: string }> }
interface EmailSender { send(msg: { to: string; subject: string; html: string; text: string }): Promise<{ id: string }> }
interface PushSender { send(tokens: string[], msg: { title: string; body: string; deepLink: string }):
  Promise<{ invalidTokens: string[] }> }
interface BillingProvider { // contractor subscriptions (FR-BIL)
  createCheckout(input: { contractorId: string; planCode: string; cycle: 'MONTHLY'|'YEARLY';
    email: string }): Promise<{ url: string; ref: string }>;
  verifyWebhook(headers: Headers, body: string): { ref: string; status: 'PAID'|'FAILED' } | null;
}
```

14.1 Name matching

CAC name vs bank account name comparison normalises both (upper-case, remove punctuation and suffixes such as LTD / LIMITED / NIG / NIGERIA / ENTERPRISES) and requires an exact match after normalisation. A near-match is sent to Platform Admin review instead of auto-failing.

15 Rewards engine

Runs only for contractors with `rewardsEnabled = true`. Points are an append-only ledger (`PointEvent`) awarded by event handlers. The unique key (`rule`, `sourceId`, `memberId`) guarantees nothing is awarded twice.

Rule	Triggered by	Awarded to
<code>TASK_ON_TIME</code>	Task confirmed, completed $\leq$ due date	Each assigned Crew Member
<code>PROOF_IN_APPROVED_STAGE</code>	Stage approved	Each staff member whose in-app media is in the stage (once per member per stage)
<code>SITE_CHECK_IN</code>	Check-in within site on a scheduled work day	That member (once per day per project)
<code>STAGE_FIRST_TIME_APPROVAL</code>	Stage approved with <code>rejectionCount = 0</code>	Site Managers assigned to the project
<code>STAGE_ON_OR_BEFORE_TARGET</code>	Stage approved and submitted $\leq$ target end	Site Managers assigned to the project
<code>ISSUE_FIXED_QUICKLY</code>	Issue fixed within a configured number of days	Site Manager who fixed it
<code>RECEIPT_CONFIRMED_ON_TIME</code>	Receipt confirmed within 3 days	Office Coordinator who confirmed
<code>CLIENT_REPLY_PROMPT</code>	Reply in project chat within a configured time of a client message	Office Coordinator
<code>DOCUMENTS_COMPLETE</code>	Project documents checklist complete	Office Coordinator
<code>RECOGNITION</code>	Manual, reason required	By Director (anyone) or Site Manager (Crew only), capped per giver per period

Configuration (TBD values) in `Contractor.rewardConfig`: points per rule, `period` (monthly / per project), `recognitionCapPerGiver`, conversion mode (`RATE` # per point, `BONUS_LEVELS`, or `CUSTOM`), leaderboard on/off. No negative points; Directors never earn points; clients never see points.

16 Billing & entitlements

ID	Rule
SR-BIL-01	Active project = status in (<code>AWAITING_CLIENT_ACCEPTANCE</code> , <code>ACTIVE</code> , <code>CLIENT_NOT_CONNECTED</code> , <code>PAUSED</code>). Drafts, completed and cancelled projects do not count.
SR-BIL-02	A new contractor's first project is flagged <code>isFreeProject</code> ; <code>freeUntil</code> = creation + 12 months. It stays free until completed or <code>freeUntil</code> , whichever is first.
SR-BIL-03	Creating/activating a second active project requires an <code>ACTIVE</code> subscription; otherwise <code>402 PLAN_REQUIRED</code> .
SR-BIL-04	Overage = $\max(0, \text{active projects} - \text{plan.includedActiveProjects})$; billed per extra project at the plan's rate (values TBD in <code>Plan</code> rows).
SR-BIL-05	Subscription <code>LAPSED</code> → contractor's active projects become read-only for the contractor (no new updates, submits, plan changes); clients keep full read access and can still upload receipts and raise disputes.
SR-BIL-06	Organisation clients: <code>earlyAccess</code> = <code>true</code> and limited to <code>RULES.orgEarlyAccessActiveProjects</code> active projects. Individual clients: no limits, always free.
SR-BIL-07	Prices, tier allowances and discounts are data (<code>Plan</code> table), editable by Platform Admin — no prices in code.

17 Security & privacy

ID	Area	Requirement
SR-SEC-01	Transport	HTTPS only; HSTS on the web app.
SR-SEC-02	Secrets	Service keys, HMAC secret, provider keys only in server environment variables; never in the mobile app or client bundles. The mobile app holds only the Supabase anon key and the user's own tokens.
SR-SEC-03	Access control	Every handler calls <code>authorize()</code> ; a test fails the build if a route handler has no authorisation call. RLS deny-all on tables (Section 6.4).
SR-SEC-04	Tokens	Invitation, share-link and viewer-link tokens are 32+ random bytes; only their SHA-256 hash is stored.
SR-SEC-05	Files	Private buckets; signed URLs expire within minutes; server re-checks size, type and hash; uploads limited per user per hour.
SR-SEC-06	Immutability	Protected records enforced by API and database triggers (Section 4.4).
SR-SEC-07	Audit	All state changes, admin actions, bank changes, role changes and permission-module changes written to <code>AuditLog</code> with actor and before/after.
SR-SEC-08	Bank safety	Director-only changes, name match, 48-hour payment pause, alerts to all connected clients by push + email + SMS, re-verification.
SR-SEC-09	Abuse	Rate limits on auth, OTP, invitations, messages, uploads; lock-out after repeated OTP failures.
SR-SEC-10	Webhooks	Supabase auth hook and billing webhooks verified by signature; replay-protected.
SR-PRV-01	Identity data	No NIN / passport numbers stored — only verification result and HMAC hash (Section 4.3).
SR-PRV-02	Minimisation	Photo EXIF stripped; precise GPS stored only for proof and check-ins; share links never include payment, bank or chat data.
SR-PRV-03	Account deletion	Personal fields anonymised (<code>User.status = DELETED</code> , name → "Deleted user", email/phone removed); project records and authored proof remain (PRD NFR-RET-02).
SR-PRV-04	NDPA	Consent captured at sign-up; privacy policy and data export on request; review before launch.

18 Quality, testing & local development

18.1 Performance targets

Measure	Target
API response (p95, excluding uploads)	< 500 ms
Mobile cold start on a low-cost Android phone	< 4 s to usable Today / Projects screen
Crew capture: tap camera → proof saved locally	< 2 s
Photo upload size after compression	≈ 300–600 KB
Chat message delivery (both online)	< 1 s

18.2 Testing strategy

Level	Tool	What
Unit	Vitest	Every state machine transition and guard in @bt/core ; progress maths; point rules; billing rules; name matching; the permission table (one test per PRD 5.3 row).
Integration	Vitest + local Supabase	API handlers against a real Postgres: happy paths, permission denials, idempotency, timers (with a controllable clock).
End-to-end (web)	Playwright	Core journeys: contractor onboarding → project → plan accepted → stage approved → payment confirmed → rating.
End-to-end (mobile)	Maestro	Crew capture offline → sync; Site Manager submit; client approve on phone.
Manual	Device lab	At least one low-end Android phone, poor network simulation, GPS off/on.

18.3 Local development

- **Supabase CLI** runs Postgres, Auth, Storage and Realtime locally (Docker); `supabase/` holds config, SQL policies and seed.
- `pnpm dev` starts the web app/API; `pnpm mobile` starts the Expo dev server.
- **Seed script**: platform presets (all project types incl. roads and bridges), a verified demo contractor with all roles, an individual client, an organisation client, and one project in every state.
- All integrations use the **fake adapters** locally; OTP codes appear in the console.
- A dev-only clock override lets you fast-forward timers (e.g. jump 3 days to test the receipt deadline).

18.4 Observability

Structured JSON logs with request id and user id; error tracking on web, API and mobile; job runner logs every job with outcome; a simple admin health panel (pending jobs, failed deliveries, failed syncs).

19 Build plan

Milestones in build order. Each ends with something testable. There is no timeline; move to the next milestone when the "done when" line is true.

#	Milestone	Scope	Done when
M0	Foundations	Monorepo, TypeScript, lint, shared packages, local Supabase, Prisma schema + first migration, RLS deny-all, fake adapters, <code>rules.ts</code> , seed.	<code>pnpm dev</code> runs web + mobile against the seeded local database.
M1	Auth & accounts	Sign-up/in with OTP (web + mobile), bootstrap, request context, <code>authorize()</code> + permission tests, devices and trust, invitations, staff and Client Reps with modules.	All roles can sign in on both apps; permission-table tests pass.
M2	Verification & directory	Contractor verification flow, admin queue, badges, bank account with name match and 48 h pause, client identity check, public directory and profiles, client↔contractor invitations.	A contractor can be verified by an admin and invited by a client.
M3	Presets, projects & plans	Platform/company presets, project wizard, plan draft/send/accept/lock, plan versions, crew assignment, client-not-connected projects, change requests.	A client accepts a plan and live stages + payment items exist.
M4	Proof capture (mobile-first)	Camera, GPS, compression, signed uploads, updates, checklist, geofence, gallery labelling, offline outbox + sync push/pull, tasks and check-ins.	Crew can capture proof offline and it syncs correctly with original time and location.
M5	Review loop	Submit with minimum-proof check, approve/reject, issues, resubmit, repeated-rejection flag, job runner, review reminders and overdue, verified/reported progress, dashboards.	Full stage cycle works on web and mobile, including day 1/3/5/7 timers.
M6	Payments & disputes	Payment schedule, due triggers, receipts, 3-day confirm / dispute, unconfirmed state, withdrawal, retention, payment and stage disputes, two-sided resolution, on-hold, 14-day flags.	Every payment and dispute path in Section 8 is covered by passing tests.
M7	Notifications	Event outbox processing, recipient rules, in-app list, push (mobile + web), email, SMS for critical, preferences, quiet hours, digest, deep links.	Every event in Appendix C reaches the right people on the right channels.
M8	Chat	Conversations, messages, attachments, voice notes, private Realtime channels, read state, offline send, in-context comments.	Staff ↔ staff and client ↔ contractor chat works live on both apps.
M9	Ratings, share links & rewards	Two-way ratings on completion, profile stats and flags, share links and crew viewer links, reward ledger, rules, recognition, periods and payouts (config values TBD).	A completed project is rated both ways; points accrue from real actions.
M10	Billing	Plans as data, free-project rules, active-project counting, overage, lapse behaviour, organisation Early Access limits, billing adapter + webhook.	Entitlement rules enforced; a fake checkout upgrades a contractor.

#	Milestone	Scope	Done when
M11	Admin console & hardening	Admin views (users, risk, subscriptions, presets, audit, tickets), rate limits, immutability triggers, account deletion, performance pass on low-end Android, E2E suites.	v1.0 feature-complete; all test suites green.
M12	Real providers & launch prep	Choose and plug in real providers behind the adapters; privacy policy / terms; app store builds. (Hosting decided here.)	Ready for a pilot with real contractors.
v1.1	Lifecycle & expenses	Pause / cancel / switch contractor with handover; ratings after cancellation; contract type; expenses; materials log.	PRD v1.1 scope complete.

20 Open items

#	Item	Where it is handled
1	Hosting / cloud provider (AWS or Azure), deployment, CI/CD	Deferred; design is host-agnostic. Decide at M12.
2	Third-party providers (verification, SMS, email, push, billing)	Deferred; adapter interfaces in Section 14; fakes until M12.
3	Reward period and recognition-point cap; points per rule	Deferred; <code>Contractor.rewardConfig</code> .
4	"Payment due before a stage starts"	Deferred; not built in v1.0; enum can be extended.
5	Subscription prices, tier allowances, overage fee, yearly discount	Business phase; stored in <code>Plan</code> rows.
6	Organisation Early Access active-project cap (3 suggested)	Business phase; <code>RULES.orgEarlyAccessActiveProjects</code> .
7	Default site radius (200 m suggested)	Confirm during M4 field testing.
8	Final product name	Parked; affects only branding, bundle ids and domains.
9	Platform preset content (stages, checklists, weights) for every project type	Write during M3, reviewed by a construction professional.

A Appendix A — Prisma schema (models)

Validated with the Prisma 7 schema engine. Enums are listed in Appendix B. The complete file (models + enums) is delivered as `schema.prisma`.

```
// BuildTrack — Prisma schema (SRD v1.0 draft)
// Conventions:
// * ids are UUIDs; User.id equals the Supabase Auth user id
// * money is stored in minor units (kobo / cents) as BigInt
// * stage weights are basis points (10000 = 100%)
// * all timestamps are UTC

generator client {
  provider = "prisma-client"
  output   = "../src/generated/prisma"
}

datasource db {
  provider = "postgresql"
  // Prisma 7: connection URLs live in prisma.config.ts (see SRD 4.1)
}

// ----- Enums -----

// v1.1
// ----- Identity -----

model User {
  id          String      @id @db.Uuid          // = Supabase Auth user id
  fullName    String
  email       String      @unique
  phone       String      @unique                // E.164
  avatarPath  String?
  timezone    String      @default("Africa/Lagos")
  quietHoursStart Int?      // minutes from midnight, local
  quietHoursEnd Int?
  isPlatformAdmin Boolean   @default(false)
  status      UserStatus   @default(ACTIVE)
  createdAt   DateTime     @default(now())
  lastLoginAt DateTime?

  devices      Device[]
  identity      IdentityVerification?
  clientMemberships ClientMember[]
  contractorMemberships ContractorMember[]
  notifications Notification[]
  notificationPrefs NotificationPreference[]
  conversationParticipants ConversationParticipant[]
}

model Device {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @db.Uuid
  user        User        @relation(fields: [userId], references: [id])
  platform    Platform
  deviceKey   String      // stable per-install id
  pushToken   String?
  trustedAt   DateTime?    // set after SMS OTP on new device
  lastSeenAt  DateTime     @default(now())
  @@unique([userId, deviceKey])
}

/// Stores the RESULT of an identity check — never the raw NIN / passport number (NDPA).
model IdentityVerification {
  id          String      @id @default(uuid()) @db.Uuid
  userId      String      @unique @db.Uuid
  user        User        @relation(fields: [userId], references: [id])
  method      IdMethod
  identityHash String      @unique                // HMAC of the ID number — enforces one account per identity
  provider    String
  providerRef  String
  status      CheckStatus
  verifiedAt   DateTime?
  createdAt   DateTime     @default(now())
}
```

```

}

// ----- Client side -----

model ClientAccount {
  id          String          @id @default(uuid()) @db.Uuid
  type        ClientAccountType
  name        String
  earlyAccess Boolean          @default(false)      // organisations only (v1)
  createdAt   DateTime         @default(now())
  members     ClientMember[]
  projects    Project[]
  ratingsReceived Rating[]    @relation("RatedClient")
}

model ClientMember {
  id          String          @id @default(uuid()) @db.Uuid
  clientAccountId String      @db.Uuid
  clientAccount ClientAccount @relation(fields: [clientAccountId], references: [id])
  userId      String          @db.Uuid
  user        User            @relation(fields: [userId], references: [id])
  role        ClientRole
  modules     ClientRepModule[] // only meaningful for CLIENT_REP
  status      MemberStatus    @default(ACTIVE)
  createdAt   DateTime         @default(now())
  projectAccess ClientRepProject[]
  @@unique([clientAccountId, userId])
}

model ClientRepProject {
  clientMemberId String      @db.Uuid
  clientMember ClientMember @relation(fields: [clientMemberId], references: [id])
  projectId      String      @db.Uuid
  project        Project      @relation(fields: [projectId], references: [id])
  @@id([clientMemberId, projectId])
}

// ----- Contractor side -----

model Contractor {
  id          String          @id @default(uuid()) @db.Uuid
  name        String
  cacName     String?
  rcNumber    String?         @unique
  tin         String?
  officeAddress String?
  state       String?
  description  String?
  projectTypes ProjectType[]
  verificationStatus VerificationStatus @default(NOT_STARTED)
  verifiedAt   DateTime?
  verificationExpiresAt DateTime?
  paymentsPausedUntil DateTime? // 48h after a bank-detail change
  rewardsEnabled Boolean       @default(false)
  rewardConfig Json?           // conversion mode, period, caps (values TBD)
  suspendedAt  DateTime?
  createdAt    DateTime        @default(now())

  members      ContractorMember[]
  submissions  VerificationSubmission[]
  badges       ContractorBadge[]
  bankAccounts BankAccount[]
  presets      StagePreset[]
  projects     Project[]
  subscription Subscription?
  ratingsReceived Rating[]    @relation("RatedContractor")
  pointEvents  PointEvent[]
  rewardPeriods RewardPeriod[]
  conversations Conversation[]
}

model ContractorMember {
  id          String          @id @default(uuid()) @db.Uuid
  contractorId String          @db.Uuid
  contractor  Contractor      @relation(fields: [contractorId], references: [id])
  userId      String          @db.Uuid
  user        User            @relation(fields: [userId], references: [id])
  role        ContractorRole
  status      MemberStatus    @default(ACTIVE)
}

```

```

    workDays      Int[]                                // 0-6, scheduled days (attendance points)
    createdAt     DateTime @default(now())
    assignments   ProjectAssignment[]
    pointEvents   PointEvent[]
    @@unique([contractorId, userId])
}

model VerificationSubmission {
    id             String @id @default(uuid()) @db.Uuid
    contractorId   String @db.Uuid
    contractor     Contractor @relation(fields: [contractorId], references: [id])
    status         VerificationStatus @default(SUBMITTED)
    submittedAt    DateTime @default(now())
    reviewedById   String? @db.Uuid
    reviewedAt     DateTime?
    rejectionReason String?
    items          VerificationItem[]
}

model VerificationItem {
    id             String @id @default(uuid()) @db.Uuid
    submissionId   String @db.Uuid
    submission     VerificationSubmission @relation(fields: [submissionId], references: [id])
    type           VerificationItemType
    status         CheckStatus @default(PENDING)
    documentPath   String?
    provider       String?
    providerRef    String?
    result         Json? // e.g. CAC name, status, incorporation date
    checkedAt      DateTime?
}

model ContractorBadge {
    id             String @id @default(uuid()) @db.Uuid
    contractorId   String @db.Uuid
    contractor     Contractor @relation(fields: [contractorId], references: [id])
    type           BadgeType
    grantedAt      DateTime @default(now())
    expiresAt      DateTime?
    @@unique([contractorId, type])
}

model BankAccount {
    id             String @id @default(uuid()) @db.Uuid
    contractorId   String @db.Uuid
    contractor     Contractor @relation(fields: [contractorId], references: [id])
    bankCode       String
    bankName       String
    accountNumber  String
    accountName    String // resolved by lookup
    nameMatchesCac Boolean
    status         BankAccountStatus @default(ACTIVE)
    createdById    String @db.Uuid
    createdAt      DateTime @default(now())
    receipts       PaymentReceipt[]
}

// ----- Invitations -----

model Invitation {
    id             String @id @default(uuid()) @db.Uuid
    type           InvitationType
    tokenHash      String @unique
    invitedById    String @db.Uuid
    clientAccountId String? @db.Uuid
    contractorId   String? @db.Uuid
    projectId      String? @db.Uuid
    targetEmail    String?
    targetPhone    String?
    targetName     String?
    role           String? // staff/rep role for STAFF / CLIENT_REP invites
    status         InvitationStatus @default(PENDING)
    expiresAt      DateTime
    createdAt      DateTime @default(now())
    acceptedAt     DateTime?
}

// ----- Presets -----

```

```

model StagePreset {
  id                String          @id @default(uuid()) @db.Uuid
  owner             PresetOwner
  contractorId      String?         @db.Uuid
  contractor        Contractor?    @relation(fields: [contractorId], references: [id])
  basedOnId         String?        @db.Uuid
  name              String
  projectType       ProjectType
  defaultPaymentStructure Json?      // company default payment structure
  isActive          Boolean         @default(true)
  createdAt         DateTime        @default(now())
  updatedAt         DateTime        @updatedAt
  stages            PresetStage[]
}

model PresetStage {
  id                String          @id @default(uuid()) @db.Uuid
  presetId          String          @db.Uuid
  preset            StagePreset    @relation(fields: [presetId], references: [id], onDelete: Cascade)
  position          Int
  name              String
  weightBp          Int
  minPhotos         Int            @default(5)
  minVideos         Int            @default(1)
  requireDescription Boolean        @default(true)
  checklist          String[]
  @@unique([presetId, position])
}

// ----- Projects & plans -----

model Project {
  id                String          @id @default(uuid()) @db.Uuid
  contractorId      String          @db.Uuid
  contractor        Contractor      @relation(fields: [contractorId], references: [id])
  clientAccountId   String?         @db.Uuid // null = client not connected
  clientAccount     ClientAccount? @relation(fields: [clientAccountId], references: [id])
  name              String
  type              ProjectType
  description        String?
  address            String
  lat               Float
  lng               Float
  siteRadiusM       Int            @default(200)
  contractValueMinor BigInt
  currency           Currency       @default(NGN)
  contractType       ContractType? // v1.1
  retentionBp        Int            @default(0)
  startDate          DateTime        @db.Date
  targetEndDate      DateTime        @db.Date
  status             ProjectStatus @default(DRAFT)
  onHold             Boolean         @default(false) // true while a dispute is open
  isFreeProject      Boolean         @default(false)
  freeUntil          DateTime?
  presetId           String?        @db.Uuid
  acceptedPlanVersionId String?    @db.Uuid
  createdById        String          @db.Uuid
  createdAt          DateTime        @default(now())
  activatedAt        DateTime?
  completedAt        DateTime?

  planVersions       PlanVersion[]
  stages             Stage[]
  assignments         ProjectAssignment[]
  repAccess           ClientRepProject[]
  updates             ProgressUpdate[]
  issues              Issue[]
  tasks              Task[]
  checkIns            CheckIn[]
  changeRequests      ChangeRequest[]
  payments            PaymentItem[]
  disputes            Dispute[]
  ratings             Rating[]
  shareLinks          ShareLink[]
  comments            Comment[]
  conversations        Conversation[]
  @@index([contractorId, status])
  @@index([clientAccountId, status])
}

```

```

model ProjectAssignment {
  projectId String @db.Uuid
  project Project @relation(fields: [projectId], references: [id])
  memberId String @db.Uuid
  member ContractorMember @relation(fields: [memberId], references: [id])
  assignedAt DateTime @default(now())
  @@id([projectId, memberId])
}

// Immutable snapshot of stages + payment structure for each version of the plan.
model PlanVersion {
  id String @id @default(uuid()) @db.Uuid
  projectId String @db.Uuid
  project Project @relation(fields: [projectId], references: [id])
  versionNo Int
  status PlanStatus @default(DRAFT)
  snapshot Json // stages, weights, dates, checklists, proof rules, payment
  items {
    clientComment String?
    sentAt DateTime?
    acceptedAt DateTime?
    acceptedById String? @db.Uuid
    fromChangeRequestId String? @db.Uuid
    createdAt DateTime @default(now())
  }
  @@unique([projectId, versionNo])
}

model Stage {
  id String @id @default(uuid()) @db.Uuid
  projectId String @db.Uuid
  project Project @relation(fields: [projectId], references: [id])
  position Int
  name String
  weightBp Int
  targetStart DateTime @db.Date
  targetEnd DateTime @db.Date
  minPhotos Int
  minVideos Int
  requireDescription Boolean
  status StageStatus @default(NOT_STARTED)
  reportedPercent Int @default(0) // contractor's estimate 0-100
  startedAt DateTime?
  submittedAt DateTime?
  approvedAt DateTime?
  rejectionCount Int @default(0)
  repeatedRejection Boolean @default(false) // set on 3rd rejection
  reviewOverdue Boolean @default(false) // 7 days without decision
  delayed Boolean @default(false) // past targetEnd
  checklist ChecklistItem[]
  reviews StageReview[]
  updates ProgressUpdate[]
  issues Issue[]
  tasks Task[]
  payments PaymentItem[]
  disputes Dispute[]
  @@unique([projectId, position])
}

model ChecklistItem {
  id String @id @default(uuid()) @db.Uuid
  stageId String @db.Uuid
  stage Stage @relation(fields: [stageId], references: [id])
  position Int
  label String
  done Boolean @default(false)
  doneById String? @db.Uuid
  doneAt DateTime?
  updates ProgressUpdate[]
  issues Issue[]
}

// ----- Proof -----

model ProgressUpdate {
  id String @id @default(uuid()) @db.Uuid
  clientRequestId String @unique // idempotency key from offline outbox
  projectId String @db.Uuid
  project Project @relation(fields: [projectId], references: [id])
}

```

```

    stageId      String      @db.Uuid
    stage        Stage       @relation(fields: [stageId], references: [id])
    checklistItemId String?   @db.Uuid
    checklistItem ChecklistItem? @relation(fields: [checklistItemId], references: [id])
    taskId       String?     @db.Uuid
    task         Task?       @relation(fields: [taskId], references: [id])
    authorId     String      @db.Uuid
    description   String?
    capturedAt    DateTime    // device time at capture (offline-safe)
    lockedAt      DateTime?   // set when the stage is submitted (immutable after)
    createdAt     DateTime    @default(now())
    media        MediaAsset[]
  }

  model MediaAsset {
    id          String      @id @default(uuid()) @db.Uuid
    kind        MediaKind
    storagePath  String      @unique
    mimeType     String
    sizeBytes    Int
    durationSec  Int?        // video/voice (video ≤ 60 s)
    width       Int?
    height      Int?
    sha256      String
    capturedInApp Boolean     // false = gallery upload (labelled, not counted)
    capturedAt   DateTime?
    lat          Float?
    lng          Float?
    gpsAccuracyM Float?
    withinSite   Boolean?    // null when no GPS
    uploadStatus UploadStatus @default(PENDING)
    uploadedById String      @db.Uuid
    createdAt    DateTime    @default(now())
    // polymorphic owners (exactly one set)
    updateId     String?     @db.Uuid
    update       ProgressUpdate? @relation(fields: [updateId], references: [id])
    messageId    String?     @db.Uuid
    message      Message?    @relation(fields: [messageId], references: [id])
    receiptId    String?     @db.Uuid
    receipt      PaymentReceipt? @relation(fields: [receiptId], references: [id])
    evidenceId   String?     @db.Uuid
    evidence      DisputeEvidence? @relation(fields: [evidenceId], references: [id])
    reviewId     String?     @db.Uuid
    review       StageReview? @relation(fields: [reviewId], references: [id])
  }

  // ----- Review, issues, tasks -----

  model StageReview {
    id          String      @id @default(uuid()) @db.Uuid
    stageId     String      @db.Uuid
    stage       Stage       @relation(fields: [stageId], references: [id])
    round       Int
    submittedById String     @db.Uuid
    submittedAt  DateTime    @default(now())
    decision     ReviewDecision?
    decidedById  String?     @db.Uuid
    decidedAt    DateTime?
    comment      String?
    failedItemIds String[]
    media        MediaAsset[]
    issues       Issue[]
    @unique([stageId, round])
  }

  model Issue {
    id          String      @id @default(uuid()) @db.Uuid
    projectId   String      @db.Uuid
    project     Project     @relation(fields: [projectId], references: [id])
    stageId     String      @db.Uuid
    stage       Stage       @relation(fields: [stageId], references: [id])
    reviewId    String      @db.Uuid
    review      StageReview @relation(fields: [reviewId], references: [id])
    checklistItemId String?  @db.Uuid
    checklistItem ChecklistItem? @relation(fields: [checklistItemId], references: [id])
    description  String
    status       IssueStatus @default(OPEN)
    assignedToId String?     @db.Uuid
    createdAt    DateTime    @default(now())
  }

```

```

    fixedAt      DateTime?
  }

  model Task {
    id            String      @id @default(uuid()) @db.Uuid
    projectId     String      @db.Uuid
    project       Project     @relation(fields: [projectId], references: [id])
    stageId      String?     @db.Uuid
    stage         Stage?     @relation(fields: [stageId], references: [id])
    title         String
    description    String?
    dueDate       DateTime?
    assigneeIds   String[]    @db.Uuid // ContractorMember ids
    status        TaskStatus @default(OPEN)
    createdById   String      @db.Uuid
    completedAt   DateTime?
    confirmedById String?     @db.Uuid
    confirmedAt   DateTime?
    createdAt     DateTime    @default(now())
    updates       ProgressUpdate[]
  }

  model CheckIn {
    id            String      @id @default(uuid()) @db.Uuid
    clientRequestId String    @unique
    projectId     String      @db.Uuid
    project       Project     @relation(fields: [projectId], references: [id])
    memberId      String      @db.Uuid
    lat           Float
    lng           Float
    gpsAccuracyM  Float
    withinSite    Boolean
    capturedAt    DateTime
    createdAt     DateTime @default(now())
  }

  // ----- Change requests -----

  model ChangeRequest {
    id            String      @id @default(uuid()) @db.Uuid
    projectId     String      @db.Uuid
    project       Project     @relation(fields: [projectId], references: [id])
    raisedBySide  Side
    raisedById    String      @db.Uuid
    title         String
    reason        String
    changes       Json // structured diff against current plan
    dateImpactDays Int        @default(0)
    amountImpactMinor BigInt   @default(0)
    status        ChangeRequestStatus @default(PENDING)
    decidedById   String?     @db.Uuid
    decidedAt     DateTime?
    createdAt     DateTime    @default(now())
    payments      PaymentItem[]
  }

  // ----- Payments -----

  model PaymentItem {
    id            String      @id @default(uuid()) @db.Uuid
    projectId     String      @db.Uuid
    project       Project     @relation(fields: [projectId], references: [id])
    type          PaymentType
    label         String
    stageId      String?     @db.Uuid
    stage         Stage?     @relation(fields: [stageId], references: [id])
    changeRequestId String?   @db.Uuid
    changeRequest ChangeRequest? @relation(fields: [changeRequestId], references: [id])
    amountMinor   BigInt      // gross
    retentionMinor BigInt      @default(0) // withheld from this payment
    currency      Currency
    dueTrigger    PaymentDueTrigger
    status        PaymentStatus @default(SCHEDULED)
    dueAt         DateTime?
    confirmDeadline DateTime? // receipt upload + 3 days
    createdAt     DateTime    @default(now())
    receipts      PaymentReceipt[]
    disputes      Dispute[]
  }

```



```

model PaymentReceipt {
    id                String          @id @default(uuid()) @db.Uuid
    paymentItemId     String          @db.Uuid
    paymentItem       PaymentItem     @relation(fields: [paymentItemId], references: [id])
    bankAccountId     String          @db.Uuid // account shown to the client when paying
    bankAccount       BankAccount     @relation(fields: [bankAccountId], references: [id])
    uploadedById     String          @db.Uuid
    transactionRef    String
    amountMinor      BigInt
    paidOn            DateTime         @db.Date
    status            ReceiptStatus   @default(ACTIVE)
    confirmedById     String?         @db.Uuid
    confirmedAt       DateTime?
    withdrawnAt       DateTime?
    uploadedAt        DateTime         @default(now())
    media             MediaAsset[]
}

// ----- Disputes -----

model Dispute {
    id                String          @id @default(uuid()) @db.Uuid
    projectId         String          @db.Uuid
    project           Project         @relation(fields: [projectId], references: [id])
    type              DisputeType
    paymentItemId     String?         @db.Uuid
    paymentItem       PaymentItem?   @relation(fields: [paymentItemId], references: [id])
    stageId           String?         @db.Uuid
    stage             Stage?         @relation(fields: [stageId], references: [id])
    openedBySide      Side
    openedById        String          @db.Uuid
    reason            String
    status            DisputeStatus   @default(OPEN)
    clientResolvedAt  DateTime?       // "Mark dispute resolved" - client side
    contractorResolvedAt DateTime?    // "Mark dispute resolved" - contractor side
    unresolvedFlaggedAt DateTime?    // set at 14 days
    resolvedAt        DateTime?
    createdAt         DateTime         @default(now())
    evidence          DisputeEvidence[]
}

model DisputeEvidence {
    id                String          @id @default(uuid()) @db.Uuid
    disputeId         String          @db.Uuid
    dispute           Dispute         @relation(fields: [disputeId], references: [id])
    side             Side
    authorId          String          @db.Uuid
    note             String
    createdAt         DateTime         @default(now())
    media            MediaAsset[]
}

// ----- Chat & comments -----

model Conversation {
    id                String          @id @default(uuid()) @db.Uuid
    type             ConversationType
    contractorId     String          @db.Uuid // every conversation is contained in one company
    contractor       Contractor      @relation(fields: [contractorId], references: [id])
    projectId        String?         @db.Uuid
    project          Project?        @relation(fields: [projectId], references: [id])
    title            String?
    createdAt        DateTime         @default(now())
    participants     ConversationParticipant[]
    messages         Message[]
}

model ConversationParticipant {
    conversationId    String          @db.Uuid
    conversation      Conversation    @relation(fields: [conversationId], references: [id])
    userId            String          @db.Uuid
    user             User            @relation(fields: [userId], references: [id])
    lastReadAt       DateTime?
    joinedAt         DateTime         @default(now())
    leftAt           DateTime?
    @@id([conversationId, userId])
}

```

```

model Message {
  id          String          @id @default(uuid()) @db.Uuid
  clientRequestId String      @unique
  conversationId String        @db.Uuid
  conversation Conversation @relation(fields: [conversationId], references: [id])
  senderId    String          @db.Uuid
  body        String?
  createdAt   DateTime         @default(now())
  media       MediaAsset[]
  @@index([conversationId, createdAt])
}

model Comment {
  id          String          @id @default(uuid()) @db.Uuid
  projectId   String          @db.Uuid
  project     Project         @relation(fields: [projectId], references: [id])
  targetType  CommentTarget
  targetId    String          @db.Uuid
  authorId    String          @db.Uuid
  body        String
  createdAt   DateTime         @default(now())
  @@index([targetType, targetId])
}

// ----- Ratings & sharing -----

model Rating {
  id          String          @id @default(uuid()) @db.Uuid
  projectId   String          @db.Uuid
  project     Project         @relation(fields: [projectId], references: [id])
  fromSide    Side
  fromUserId  String          @db.Uuid
  ratedContractorId String?   @db.Uuid
  ratedContractor Contractor? @relation("RatedContractor", fields: [ratedContractorId], references: [id])
  ratedClientId String?       @db.Uuid
  ratedClient  ClientAccount? @relation("RatedClient", fields: [ratedClientId], references: [id])
  stars        Int             // 1-5
  review       String?
  response     String?         // one public response by the rated side
  createdAt   DateTime         @default(now())
  @@unique([projectId, fromSide])
}

model ShareLink {
  id          String          @id @default(uuid()) @db.Uuid
  projectId   String          @db.Uuid
  project     Project         @relation(fields: [projectId], references: [id])
  kind        ShareLinkKind
  tokenHash   String          @unique
  createdById String          @db.Uuid
  label       String?
  expiresAt   DateTime?
  revokedAt   DateTime?
  createdAt   DateTime         @default(now())
}

// ----- Rewards -----

model PointEvent {
  id          String          @id @default(uuid()) @db.Uuid
  contractorId String          @db.Uuid
  contractor  Contractor      @relation(fields: [contractorId], references: [id])
  memberId    String          @db.Uuid
  member      ContractorMember @relation(fields: [memberId], references: [id])
  projectId   String?         @db.Uuid
  rule        PointRule
  points      Int              // always > 0 (no negative points)
  sourceId    String           // task/stage/receipt/check-in/etc.
  awardedById String?          @db.Uuid      // RECOGNITION only
  reason       String?
  createdAt   DateTime         @default(now())
  @@unique([rule, sourceId, memberId])      // never award twice for the same thing
}

model RewardPeriod {
  id          String          @id @default(uuid()) @db.Uuid
  contractorId String          @db.Uuid
  contractor  Contractor      @relation(fields: [contractorId], references: [id])
  startsAt    DateTime

```

```

    endsAt      DateTime
    closedAt    DateTime?
    payouts     RewardPayout[]
}

model RewardPayout {
    id          String      @id @default(uuid()) @db.Uuid
    periodId    String      @db.Uuid
    period      RewardPeriod @relation(fields: [periodId], references: [id])
    memberId    String      @db.Uuid
    points      Int
    amountMinor BigInt?
    paidAt      DateTime?
    markedById  String?     @db.Uuid
    @unique([periodId, memberId])
}

// ----- Billing -----

model Plan {
    code          String      @id                    // e.g. STARTER, GROWTH, PRO (prices TBD)
    name          String
    includedActiveProjects Int?                    // null = unlimited
    monthlyPriceMinor BigInt?
    yearlyPriceMinor BigInt?
    extraProjectPriceMinor BigInt?
    isActive      Boolean      @default(true)
    subscriptions Subscription[]
}

model Subscription {
    id          String      @id @default(uuid()) @db.Uuid
    contractorId String      @db.Uuid
    contractor   Contractor  @relation(fields: [contractorId], references: [id])
    planCode     String?
    plan         Plan?       @relation(fields: [planCode], references: [code])
    status       SubscriptionStatus @default(FREE_PROJECT)
    cycle        BillingCycle?
    currentPeriodEnd DateTime?
    freeProjectId String?     @db.Uuid
    freeProjectEndsAt DateTime? // finish or 12 months, whichever first
    providerRef  String?
    createdAt    DateTime     @default(now())
}

// ----- Notifications & jobs -----

model Notification {
    id          String      @id @default(uuid()) @db.Uuid
    userId      String      @db.Uuid
    user        User        @relation(fields: [userId], references: [id])
    eventType   String      // e.g. STAGE_READY_FOR_REVIEW (see catalogue)
    title       String
    body        String
    deeplink    String      // opens the exact item
    critical    Boolean      @default(false)
    readAt      DateTime?
    createdAt   DateTime     @default(now())
    deliveries  NotificationDelivery[]
    @index([userId, readAt])
}

model NotificationDelivery {
    id          String      @id @default(uuid()) @db.Uuid
    notificationId String      @db.Uuid
    notification Notification  @relation(fields: [notificationId], references: [id])
    channel     Channel
    status      DeliveryStatus @default(QUEUED)
    attempts    Int           @default(0)
    lastError   String?
    sentAt      DateTime?
}

model NotificationPreference {
    userId      String      @db.Uuid
    user        User        @relation(fields: [userId], references: [id])
    eventType   String
    push        Boolean      @default(true)
    email       Boolean      @default(true)
}

```

```

    sms          Boolean @default(true)
    digest        Boolean @default(false)           // progress updates only
    @id([userId, eventType])
}

/// Transactional outbox: domain events written in the same DB transaction as the change.
model DomainEvent {
    id            String      @id @default(uuid()) @db.Uuid
    type          String
    payload        Json
    createdAt      DateTime @default(now())
    processedAt    DateTime?
    @@index([processedAt])
}

/// Timers: review reminders, confirm deadlines, 14-day flags, payment-pause end, free-project end.
model ScheduledJob {
    id            String      @id @default(uuid()) @db.Uuid
    type          String
    runAt         DateTime
    payload        Json
    dedupeKey     String      @unique
    status        JobStatus @default(PENDING)
    attempts      Int         @default(0)
    lastError     String?
    createdAt      DateTime @default(now())
    @@index([status, runAt])
}

// ----- Admin -----

model AuditLog {
    id            String      @id @default(uuid()) @db.Uuid
    actorId       String?     @db.Uuid
    action        String
    entityType     String
    entityId       String
    before        Json?
    after         Json?
    ip            String?
    createdAt      DateTime @default(now())
    @@index([entityType, entityId])
}

model SupportTicket {
    id            String      @id @default(uuid()) @db.Uuid
    userId        String      @db.Uuid
    subject       String
    body          String
    status        TicketStatus @default(OPEN)
    assignedTo    String?     @db.Uuid
    createdAt      DateTime @default(now())
    closedAt      DateTime?
}

```

B Appendix B — Enums

Enum	Values
UserStatus	ACTIVE · SUSPENDED · DELETED
Platform	ANDROID · IOS · WEB
IdMethod	NIN · PASSPORT
CheckStatus	PENDING · PASSED · FAILED
ClientAccountType	INDIVIDUAL · ORGANISATION
ClientRole	CLIENT_ADMIN · CLIENT_REP
ClientRepModule	VIEW_PROGRESS · REVIEW_STAGES · PAYMENTS · CHANGE_REQUESTS · MESSAGE_CONTRACTOR · SHARE_LINKS · INVITE_PEOPLE
ContractorRole	DIRECTOR · SITE_MANAGER · OFFICE_COORDINATOR · CREW_MEMBER
MemberStatus	INVITED · ACTIVE · REMOVED
VerificationStatus	NOT_STARTED · SUBMITTED · UNDER_REVIEW · VERIFIED · REJECTED · EXPIRED
VerificationItemType	CAC · TIN · DIRECTOR_ID · BANK_NAME_MATCH · ADDRESS_PROOF · PHONE · EMAIL · COREN · CORBON · BPP · TAX_CLEARANCE · INSURANCE · PORTFOLIO
BadgeType	REGISTERED_ENGINEER · REGISTERED_BUILDER · GOVERNMENT_READY · TAX_CLEARED · INSURED · PORTFOLIO
BankAccountStatus	ACTIVE · SUPERSEDED
InvitationType	CLIENT_INVITES_CONTRACTOR · CONTRACTOR_INVITES_CLIENT · STAFF · CLIENT_REP
InvitationStatus	PENDING · ACCEPTED · DECLINED · EXPIRED · CANCELLED
PresetOwner	PLATFORM · CONTRACTOR
ProjectType	BUNGALOW · DUPLEX · STOREY_BUILDING · COMMERCIAL · RENOVATION · ROAD · BRIDGE · OTHER
Currency	NGN · USD · GBP
ProjectStatus	DRAFT · AWAITING_CLIENT_ACCEPTANCE · CLIENT_NOT_CONNECTED · ACTIVE · PAUSED · COMPLETED · CANCELLED
ContractType	FIXED_PRICE · OPEN_BOOK
PlanStatus	DRAFT · SENT · CHANGES_REQUESTED · ACCEPTED · SUPERSEDED
StageStatus	NOT_STARTED · IN_PROGRESS · READY_FOR_REVIEW · APPROVED
ReviewDecision	APPROVED · REJECTED
IssueStatus	OPEN · FIXED · CLOSED
TaskStatus	OPEN · DONE_PENDING_CONFIRMATION · CONFIRMED · CANCELLED
MediaKind	PHOTO · VIDEO · VOICE · DOCUMENT
UploadStatus	PENDING · UPLOADED · FAILED
Side	CLIENT · CONTRACTOR
ChangeRequestStatus	PENDING · APPROVED · REJECTED · WITHDRAWN
PaymentType	DEPOSIT · STAGE · CHANGE_REQUEST · RETENTION_RELEASE
PaymentDueTrigger	BEFORE_FIRST_STAGE · ON_STAGE_APPROVAL · ON_CHANGE_APPROVAL · ON_HANDOVER

Enum	Values
PaymentStatus	SCHEDULED · DUE · RECEIPT_UPLOADED · CONFIRMED · UNCONFIRMED · DISPUTED
ReceiptStatus	ACTIVE · WITHDRAWN · CONFIRMED · DISPUTED
DisputeType	PAYMENT · STAGE
DisputeStatus	OPEN · RESOLVED
ConversationType	COMPANY_DIRECT · COMPANY_GROUP · PROJECT
CommentTarget	STAGE · UPDATE · ISSUE · PAYMENT · CHANGE_REQUEST
Channel	IN_APP · PUSH · EMAIL · SMS
DeliveryStatus	QUEUED · SENT · FAILED · SKIPPED
JobStatus	PENDING · RUNNING · DONE · FAILED · CANCELLED
ShareLinkKind	PROJECT_SHARE · CREW_VIEWER
PointRule	TASK_ON_TIME · PROOF_IN_APPROVED_STAGE · SITE_CHECK_IN · STAGE_FIRST_TIME_APPROVAL · STAGE_ON_OR_BEFORE_TARGET · ISSUE_FIXED_QUICKLY · RECEIPT_CONFIRMED_ON_TIME · CLIENT_REPLY_PROMPT · DOCUMENTS_COMPLETE · RECOGNITION
SubscriptionStatus	FREE_PROJECT · ACTIVE · PAST_DUE · LAPSED · CANCELLED
BillingCycle	MONTHLY · YEARLY
TicketStatus	OPEN · PENDING · CLOSED

C Appendix C — Notification event catalogue

In-app is always on. ● = sent by default. Critical events (dispute, bank change, OTP, new device) cannot be disabled.

Event code	Trigger	Recipients	Push	Email	SMS
PLAN_SENT	Plan / payment structure sent	Client Admin	●	●	
PLAN_ACCEPTED / PLAN_CHANGES_REQUESTED	Client decided on plan	Director, SM	●	●	
STAGE_READY_FOR_REVIEW	Stage submitted	C.Admin, Rep(REVIEW)	●	●	
REVIEW_REMINDER (day 1)	Timer	C.Admin, Rep(REVIEW)	●		
REVIEW_REMINDER (day 3)	Timer	C.Admin, Rep(REVIEW)	●	●	
REVIEW_REMINDER (day 5)	Timer	C.Admin, Rep(REVIEW)	●	●	●
STAGE_REVIEW_OVERDUE	Timer (day 7)	Client reviewers; Director, SM	●	● (client)	
STAGE_APPROVED / STAGE_REJECTED	Review decision	Director, SM	●	●	
STAGE_REPEATED_REJECTION	3rd rejection	Both sides	●	●	
STAGE_RESUBMITTED	Issues fixed, resubmitted	C.Admin, Rep(REVIEW)	●		
UPDATE_POSTED	New progress update	C.Admin, Rep(VIEW) — digest option	●		
PAYMENT_DUE	Trigger met	C.Admin, Rep(PAYMENTS)	●	●	
RECEIPT_UPLOADED	Client uploaded receipt	Director, OC	●	●	
RECEIPT_REMINDER	Timer (day 2)	Director, OC	●	●	●
PAYMENT_CONFIRMED / PAYMENT_DISPUTED	Contractor response	C.Admin, Rep(PAYMENTS)	●	●	
PAYMENT_UNCONFIRMED	Timer (day 3)	C.Admin, Rep(PAYMENTS)	●	●	
CHANGE_REQUEST_RAISED / _DECIDED	Change request	Other side (per module)	●	●	
DISPUTE_OPENED	Dispute opened	Both sides (involved)	●	●	●
DISPUTE_UNRESOLVED	Timer (day 14)	Both sides (involved)	●	●	●
DISPUTE_RESOLVED	Both sides resolved	Both sides (involved)	●	●	
BANK_DETAILS_CHANGED	Director changed account	All connected clients (Admin + Rep(PAYMENTS))	●	●	●
PAYMENTS_RESUMED	Timer (48 h)	Clients with due payments	●		
INVITATION_RECEIVED / ACCEPTED	Invitations	Invitee / inviter	●	●	
ASSIGNED_TO_PROJECT / TASK_ASSIGNED	Crew assignment	Staff / Crew	●		
POINTS_AWARDED / BONUS_PAID	Rewards	Staff / Crew	●		
RATING_RECEIVED / RATING_REQUESTED	Ratings	Rated side / both at completion	●	● (request)	

Event code	Trigger	Recipients	Push	Email	SMS
VERIFICATION_DECIDED / EXPIRING	Verification	Director	●	●	
SUBSCRIPTION_DUE / FAILED / FREE_ENDING	Billing	Director	●	●	
CHAT_MESSAGE	New message	Participants	●		
NEW_DEVICE_LOGIN / OTP	Security	User			●

D Appendix D — Error codes

HTTP	Code	Meaning
400	VALIDATION_FAILED	Input failed Zod validation; <code>details</code> lists fields.
401	UNAUTHENTICATED	Missing / expired token.
401	DEVICE_NOT_TRUSTED	New device must complete SMS OTP.
402	PLAN_REQUIRED	Action needs an active subscription (e.g. second active project).
403	FORBIDDEN	Role or Client Rep module does not allow this action.
403	CONTRACTOR_NOT_VERIFIED	Contractor must be verified first.
403	COREN_REQUIRED	Road / bridge project needs COREN registration.
404	NOT_FOUND	Resource missing or not visible to the caller.
409	INVALID_STATE	Transition not allowed from the current state; <code>details.current</code> returned.
409	DUPLICATE_IDENTITY	Identity already linked to another client account.
409	PROJECT_ON_HOLD	A dispute is open; action blocked.
422	MIN_PROOF_NOT_MET	Stage cannot be submitted; <code>details</code> shows counts vs minimum.
422	WEIGHTS_INVALID	Stage weights do not total 100 %.
422	BANK_NAME_MISMATCH	Account name does not match the CAC name.
422	MEDIA_INVALID	Wrong type, too large, hash mismatch or video over 60 s.
423	PAYMENTS_PAUSED	Within 48 h of a bank-detail change.
423	READ_ONLY	Subscription lapsed or free project ended (contractor side).
429	RATE_LIMITED	Too many requests; retry after <code>Retry-After</code> .
500	INTERNAL	Unexpected error; logged with request id.

— End of document —